

누리로봇 제어 프로토콜 메뉴얼

온라인 메뉴얼 URL: <https://manuals.nurirobot.info/latest/rs485/index.html>

적용대상: 누리로봇 RS485 기반 제어기 / 액추에이터 전 제품군

1. 문서개요

1.1 목적

RS485 기반 누리로봇 제품 제어를 위한 통신 프로토콜을 명확히 이해하고 적용할 수 있도록 구조, 명령 체계, 피드백 구조를 정의한다.

1.2 적용 범위

본 문서는 아래 제품에 공통적으로 적용된다.

- BLDC 모터 컨트롤러(RS485 통신)
 - BCA3508
 - BCA3516
 - BCA3516E
 - BCA5008
 - BCA5016
 - BCA5016E
 - DC 모터 컨트롤러(RS485 통신)
 - DCA3508
 - DCA5008
 - 스마트모터(RS485 통신)
 - SA 모터군
 - SA59
 - SA70
 - SA84
 - SA112
 - SB 모터군
 - SB60
 - SC 모터군
 - SC70(SM70)
-

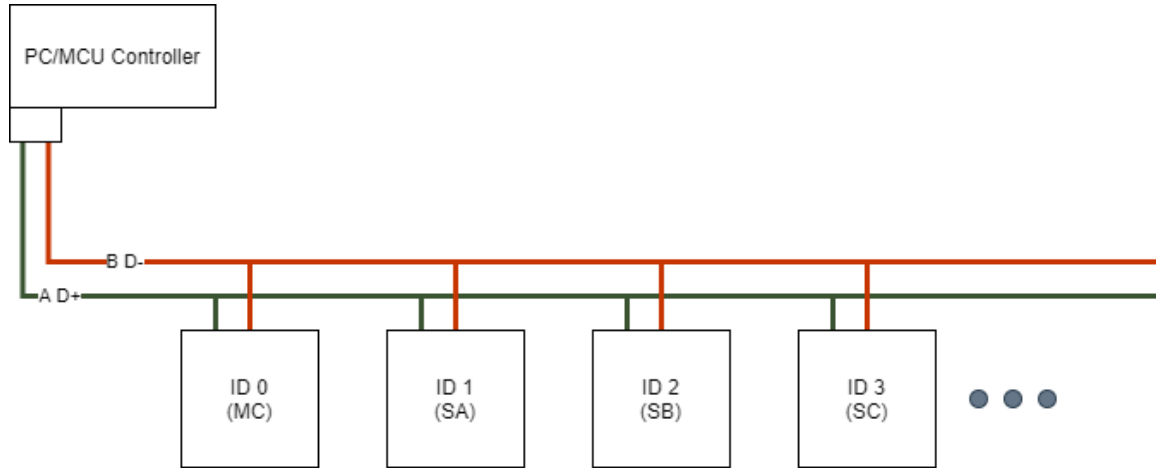
2. 시스템 구성(System Architecture)

2.1 통신 개요

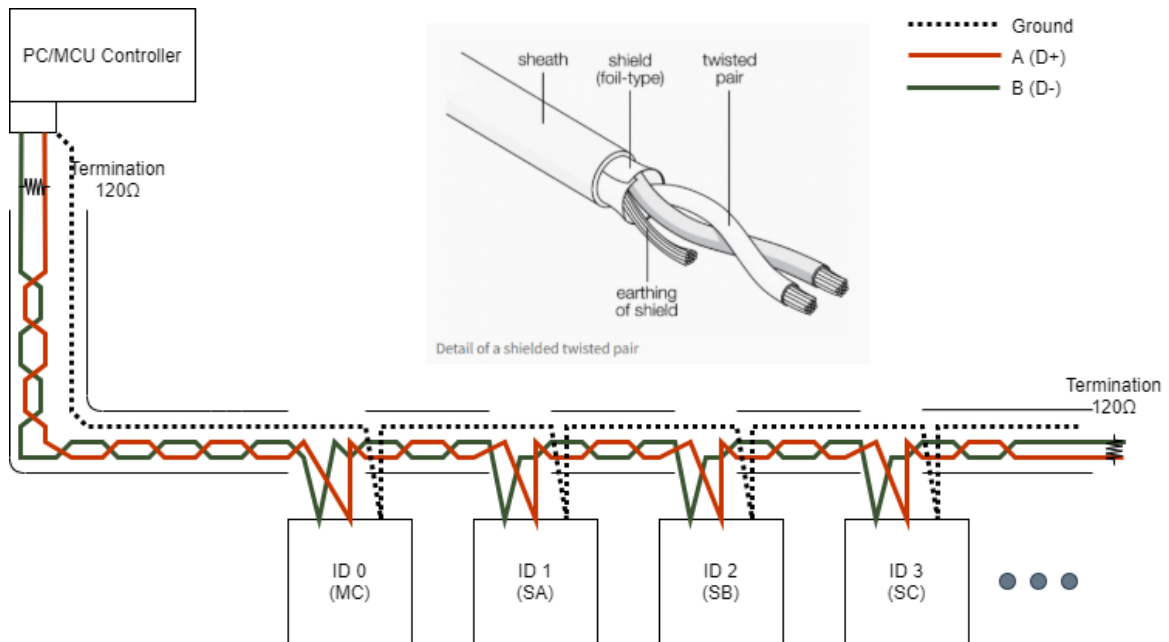
- RS485 Half-Duplex
- Data bit: 8bit / Stop bit: 1bit / Parity: None
- Baudrate: 1,200 ~ 1,000,000bps (출고 기본값: 9600bps)
- 통신 구조: 1:N Master-Slave

2.2 네트워크 구성 예시

구성 예시



구성 권장



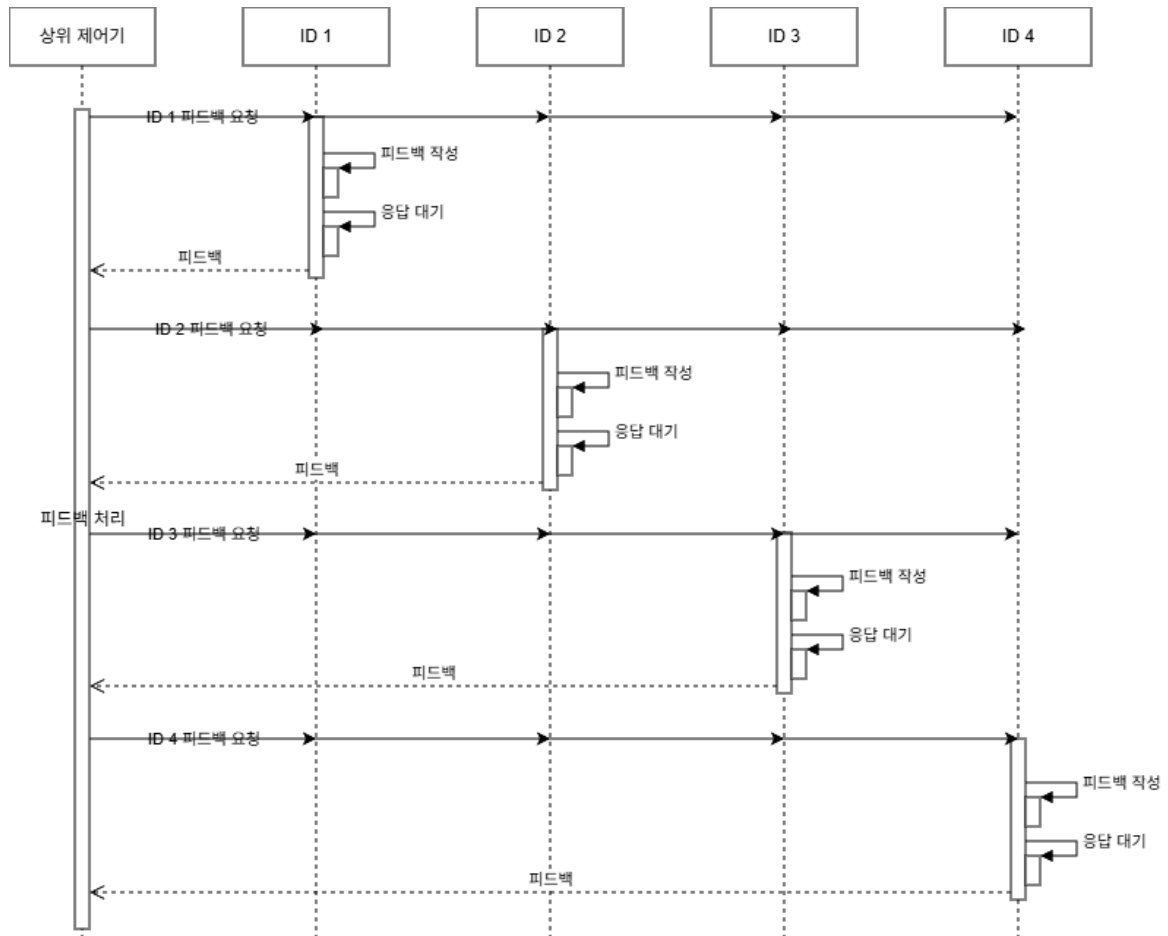
- Daisy-chain 방식 권장
- 종단저항 반드시 양 끝단 120Ω 적용
- Shielded Cable 권장
- 모든 모듈의 GND 공통 연결

※ 제품(모터제어기, 스마트모터) 구성 상 전원부와 통신부가 2개씩으로 구성되어 있으나 두개 모두 사용하는 것을 권장하지 않음.

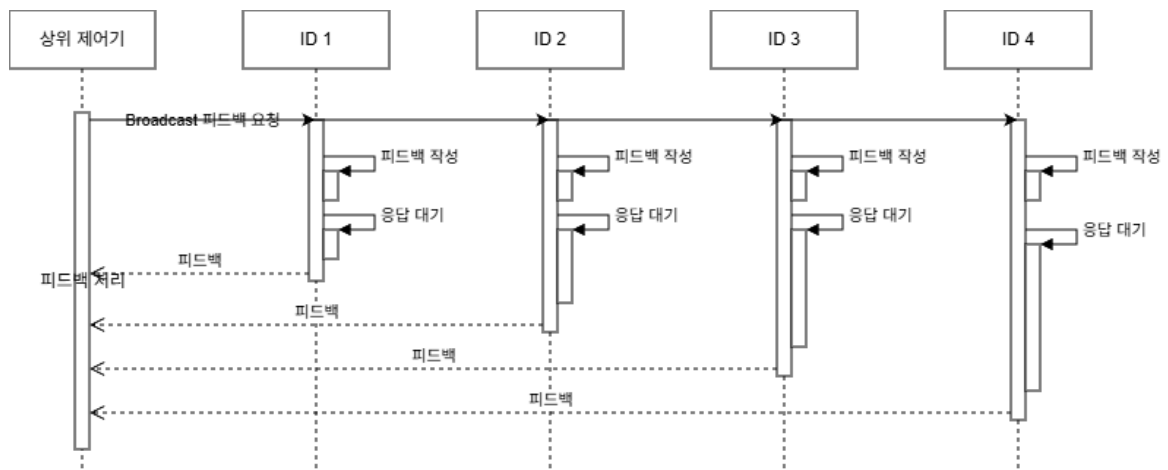
2.3 Broadcast 사용 주의사항

- ID 0xFF는 Broadcast 전용
- Broadcast 사용 시 피드백 요청 금지
- 단, 응답시간 차등 설정 시 동시 Broadcast 피드백 가능
 - 예) 통신속도가 115200bps 일 때, ID 1번 응답시간(us) 1(100us) 설정, ID 2번 응답시간(us) 21 (2100us = 2.1ms), ID 3번 응답시간(us) 42 (4200us = 4.2ms), ID 4번 응답시간 63(6300us = 6.3ms)으로 설정할 경우 위치피드백이나 속도피드백을 Broadcast 방식으로 1회 호출하여, 4대의 장비의 피드백 수신이 가능하다.

일반적인 다중 피드백



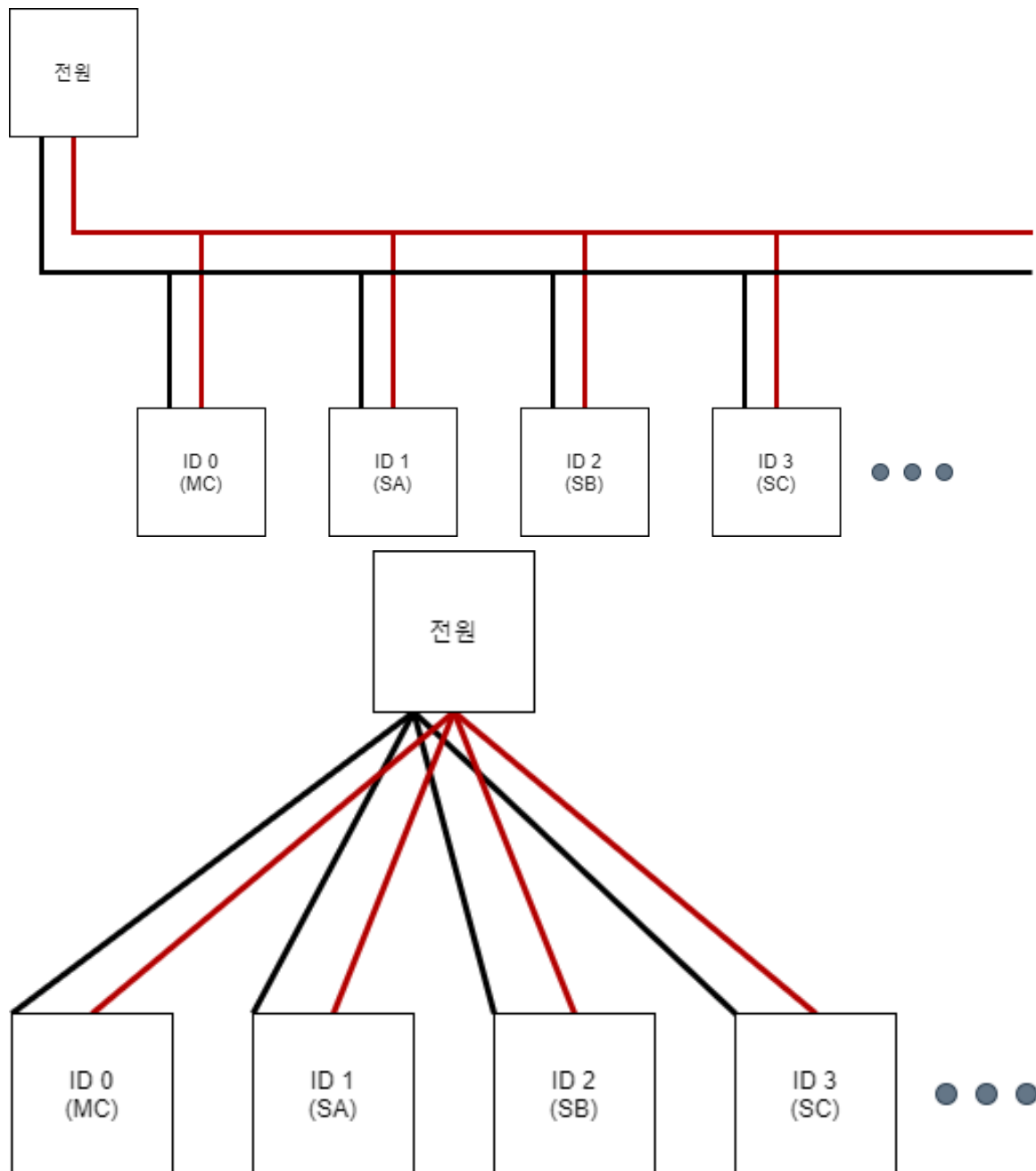
Broadcast 피드백



2.4 DC 전원 배선

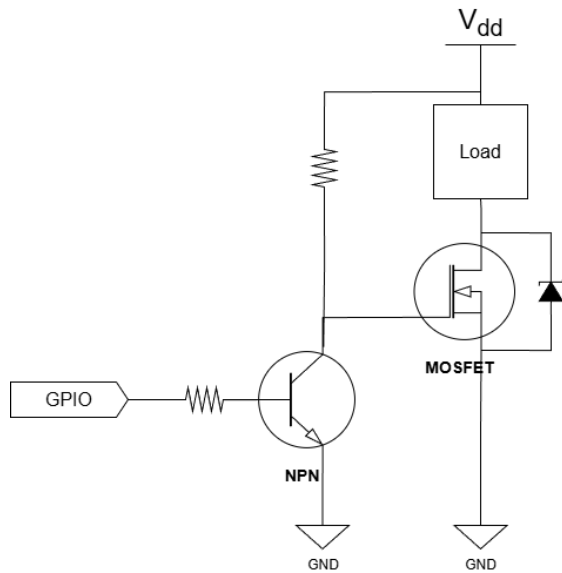
- 연결방식

- 병렬
- 버스형

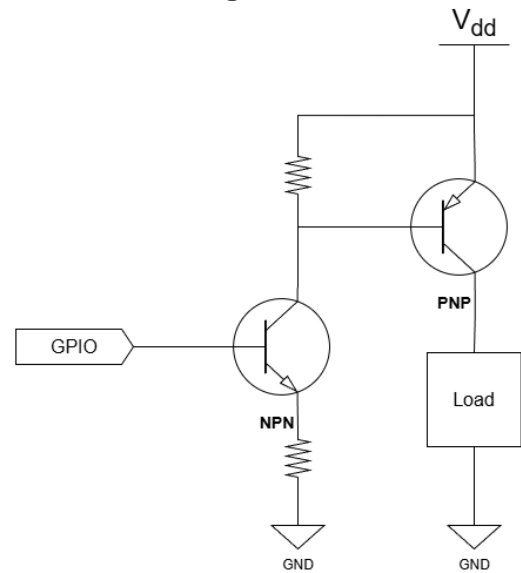


- 반드시 모든 노드의 전원 GND를 공통으로 연결
- 전원 제어를 LowSide Switch 제어를 할 경우 통신선·신호선을 통해 역전류 유입 통신 IC 손상.
- 전원 제어는 HighSide Switch 제어로 구성하여야 한다.

Low Side Switch



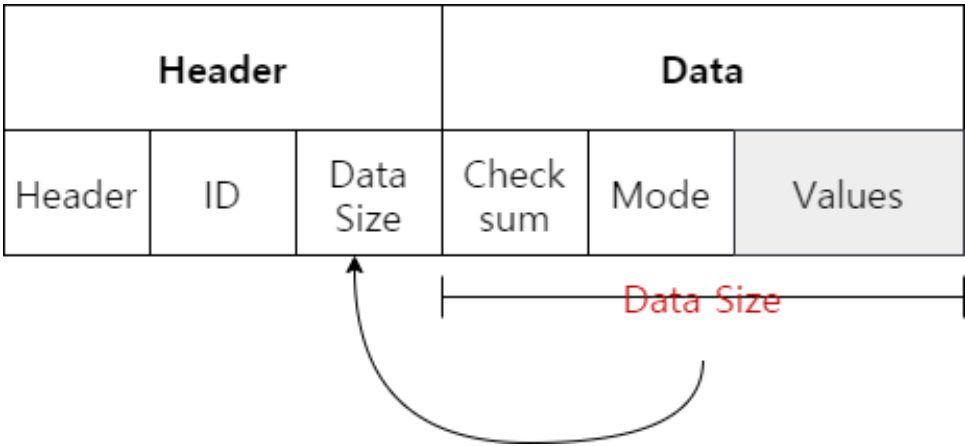
High Side Switch



- Low Side Switch는 모든 그라운드 공유된 제품이 동시에 전원이 꺼질 경우, 외부의 별도 전원을 쓰는 장치와 연결이 되지 않을 경우에만 안전하다.
- Low Side Switch가 비용적 이득이 있음.
- Low Side Switch를 릴레이 스위치 제어용으로 사용하고 릴레이 스위치를 전원 제어로 사용하면 전류 이슈 등에서 자유로울 수 있다.

3. 데이터 프레임 구조 (Packet Structure)

필드명	크기(Byte)	Index	설명
Header	2	[0-1]	Header (0xFFFE)
ID	1	[2]	ID (0x00~0xFE, 0xFF=Broadcast)
Data Size	1	[3]	Data Size (N)
Check Sum	1	[4]	Checksum = ~((ID + Size + Mode + Data...) & 0xFF)
Mode	1	[5]	명령 구분 코드
Values	n	[6..]	명령에 따른 데이터 영역



3.1 Checksum 계산식

Checksum = ~((ID + DataSize + Mode + Data...]) & 0xFF)

3.2 송수신 예시

TX : 0xFF 0xFE 0x00 0x02 0x5C 0xA1

RX : 0xFF 0xFE 0x00 0x08 0x26 0xD1 0x00 0x00 0x00 0x00 0x00 0x00

4. 명령 체계 (Command Set Overview)

4.1 명령 분류

구분	Mode 범위	설명
제어 명령	0x01~0x03	위치, 속도, 가감속 제어
설정 명령	0x04~0x0F	제어기 설정, 통신 설정, ID, 감속비 등
특수 명령	0x10~0x11	공장 초기화, Open-loop 제어
요청 명령	0xA0~0xAA, 0xCD	피드백 요청
응답 명령	0xD0~0xFD	장치 피드백 및 상태 보고

PC(MCU) → 컨트롤러

구분	Mode(hex)	BCA, DCA	SA, SB, SC	SM(구형 SC)
1	0x01	위치, 속도 제어	위치, 속도 제어	지원안함
2	0x02	가감속 위치 제어	가감속 위치 제어	지원안함
3	0x03	가감속 속도 제어	가감속 속도 제어	가감속 속도 제어
4	0x04	위치제어기 설정	위치제어기 설정	지원안함
5	0x05	속도제어기 설정	속도제어기 설정	속도제어기 설정
6	0x06	ID 설정	ID 설정	외부 감속비 설정
7	0x07	통신속도 설정	통신속도 설정	제어 On/Off설정
8	0x08	통신 응답시간 설정	통신 응답시간 설정	위치 초기화
9	0x09	모터 정격속도 설정	외부 감속비 설정	공장 초기화
10	0x0A	분해능 설정	제어 On/Off설정	지원안함
11	0x0B	감속비 설정	위치제어 모드 설정	지원안함
12	0x0C	제어 On/Off 설정	위치초기화	지원안함
13	0x0D	위치제어 모드 설정	공장 초기화	지원안함
14	0x0E	제어방향 설정	Echo 모드 설정	지원안함
15	0x0F	위치 초기화	지원안함	지원안함
16	0x10	공장 초기화	지원안함	지원안함
17	0x11	Open-Loop 구동	Open-Loop 구동	지원안함

구분	Mode(hex)	BCA, DCA	SA, SB, SC	SM(구형 SC)
18	0xA0	Ping 요청	Ping 요청	Ping 요청
19	0xA1	위치 피드백 요청	위치 피드백 요청	속도 피드백 요청
20	0xA2	속도 피드백 요청	속도 피드백 요청	속도제어기 피드백
21	0xA3	위치제어기 피드백 요청	위치제어기 피드백 요청	통신 응답시간 피드백
22	0xA4	속도제어기 피드백 요청	속도제어기 피드백 요청	외부 감속비 피드백
23	0xA5	통신 응답시간 피드백 요청	통신 응답시간 피드백 요청	제어 On/Off 피드백
24	0xA6	모터 정격속도 피드백	외부 감속비 피드백	펌웨어 버전 피드백
25	0xA7	분해능 피드백	제어 On/Off 피드백	지원안함
26	0xA8	감속비 피드백	위치제어 모드 피드백	지원안함
27	0xA9	제어 On/Off 피드백	절대엔코더 피드백	지원안함

28	0xAA	위치제어 모드 피드백	Echo 모드 피드백	지원안함
29	0xAB	제어방향 피드백	지원안함	지원안함
30	0xCD	펌웨어 버전	펌웨어 버전	펌웨어 버전

컨트롤러 → PC(MCU)

구분	Mode(hex)	BCA, DCA	SA, SB, SC	SM(구형 SC)
31	0xD0	Ping 응답	Ping 응답	Ping 응답
32	0xD1	위치 피드백 응답	위치 피드백 응답	속도 피드백 응답
33	0xD2	속도 피드백 응답	속도 피드백 응답	속도제어기 피드백 응답
34	0xD3	위치제어기 피드백 응답	위치제어기 피드백 응답	통신 응답시간 피드백 응답
35	0xD4	속도제어기 피드백 응답	속도제어기 피드백 응답	외부 감속비 피드백 응답
36	0xD5	통신 응답시간 피드백 응답	통신 응답시간 피드백 응답	제어 On/Off 피드백 응답
37	0xD6	모터 정격속도 피드백 응답	외부 감속비 피드백 응답	펌웨어 버전 피드백 응답
38	0xD7	분해능 피드백 응답	제어 On/Off 피드백 응답	지원안함
39	0xD8	감속비 피드백 응답	위치제어 모드 피드백 응답	지원안함
40	0xD9	제어 On/Off 피드백 응답	절대엔코더 피드백 응답	지원안함
41	0xDA	위치제어 모드 피드백 응답	Echo 모드 피드백 응답	지원안함
42	0xDB	제어방향 피드백 응답	지원안함	지원안함
43	0xFD	펌웨어 버전 응답	펌웨어 버전 응답	펌웨어 버전 응답

- ※ 설정 명령은 기기별로 모드 구분자가 다르다.
- ※ BCA와 DCA는 동일하지만, SA, SB, SC는 설정 명령이 다름.

4.2 제품별 지원 명령 요약

제품	지원 범위	특이 사항
BCA, DCA	0x01~0x11	분해능, 감속비, 제어방향 포함, Open-loop PWM 제어 포함
SA, SB, SC	0x01~0x0E, 0x11	Open-loop PWM 제어 포함, Echo 모드 설정

- ※ 펌웨어 버전에 따라 일부 지원되지 않는 기능이 존재한다.

5. 제어 명령 (Control Commands)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data				
			Check sum [1byte]	Mode [1byte]	위치방향 [1byte]	위치 [2byte]	속도 [2byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x07	부록 참고	0x01 (위치, 속도제어)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0xFFFF (0 ~ 65533) [0.01Degree]	0x0001 ~ 0xFFFF (1 ~ 65533) [0.1RPM]

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data				
			Check sum [1byte]	Mode [1byte]	위치방향 [1byte]	위치 [2byte]	위치 도달 시간 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x06	부록 참고	0x02 (가감속 위치제어)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0xFFFF (0 ~ 65533) [0.01Degree]	0x01 ~ 0xFF (1 ~ 255) [0.1s]

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data				
			Check sum [1byte]	Mode [1byte]	위치방향 [1byte]	위치 [2byte]	위치 도달 시간 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x06	부록 참고	0x02 (가감속 위치제어)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0xFFFF (0 ~ 65533) [0.01Degree]	0x01 ~ 0xFF (1 ~ 255) [0.1s]

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data			
			Check sum [1byte]	Mode [1byte]	구동방향 [1byte]	PWM 듀티비 [2byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x05	부록 참고	0x11 (Open-Loop 구동)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0x2710 (0 ~ 10000) [0.01%]

Mode	명령명	데이터 구조	단위	설명
0x01	위치-속도제어	[방향, 위치, 속도]	deg(0.01), RPM(0.1)	위치 및 속도를 동시에 지정
0x02	가감속 위치제어	[방향, 위치, 도달시간]	sec(0.1)	도달 시간 기반 위치 이동
0x03	가감속 속도제어	[방향, 속도, 도달시간]	sec(0.1)	도달 시간 기반 속도 변경
0x11	Open-loop 구동	[방향, 듀티비]	0.01%	PWM 방식의 개방 루프 제어

방향: 1 bytes, 위치: 2 bytes, 속도: 2 bytes, 듀티비: 2 bytes

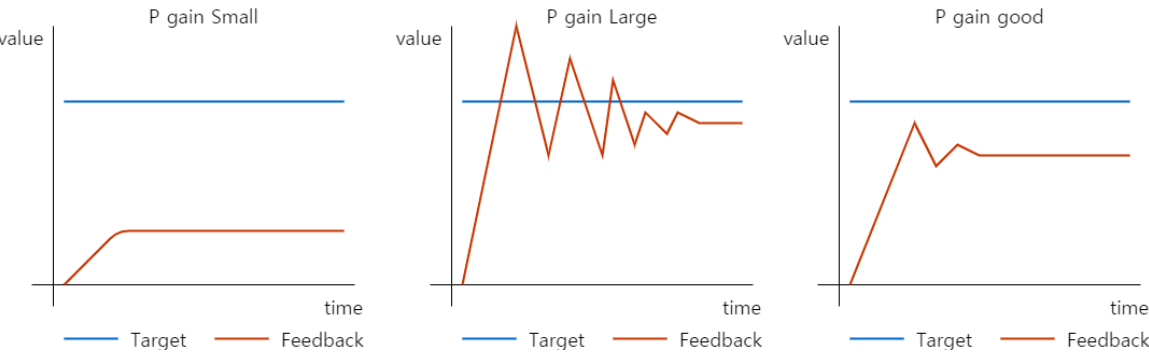
5.1 PID 게인 튜닝

- PID 게인 튜닝은 목표 도달 방식에 따라 결정되어야 한다.
- 한번에 목표에 도달하는 것이 목적일 경우 PID 순으로 튜닝하는 것이 유리하다.
 - 위치 제어
- 목표에 빠르게 도달하는 것이 목적일 경우 PDI 순으로 튜닝하는 것이 유리하다.
 - 속도 제어
- Kp, Ki, Kd: 1byte 정규화 값(0~255)
- 실제 제어식은 제품 내부 알고리즘에 의해 스케일링됨
- PID 튜닝 그래프는 개념 참고용이며 절대 scale은 아님

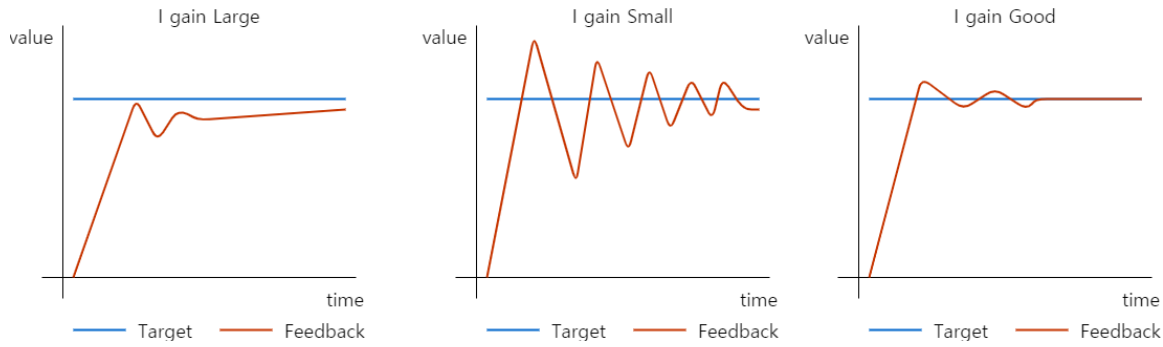
5.1.1 PID 순차 튜닝

- 목표에 부드럽게 도달할 수 있다.

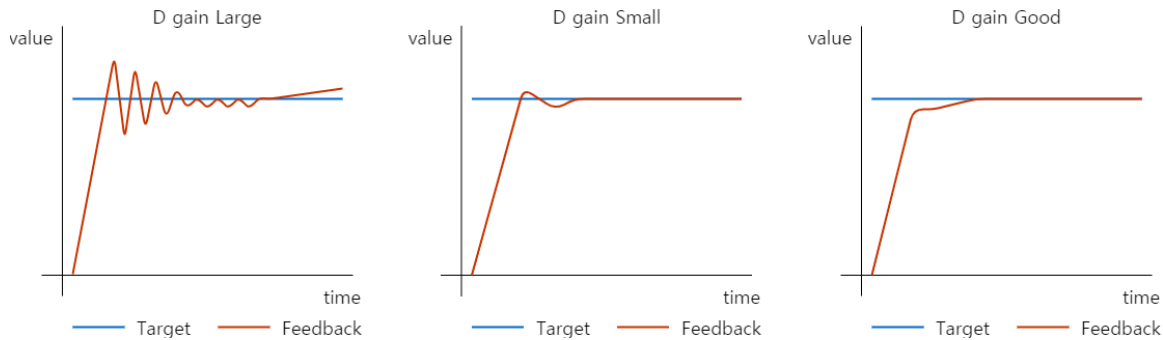
P 게인 튜닝



P+I 게인 튜닝

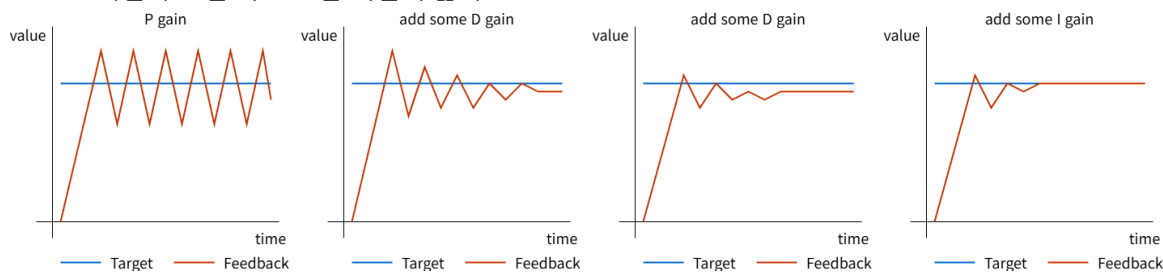


P+I+D 게인 튜닝



5.1.2 P D I 순차 튜닝

- P 게인이 크면 목표 도달 시간이 짧다.

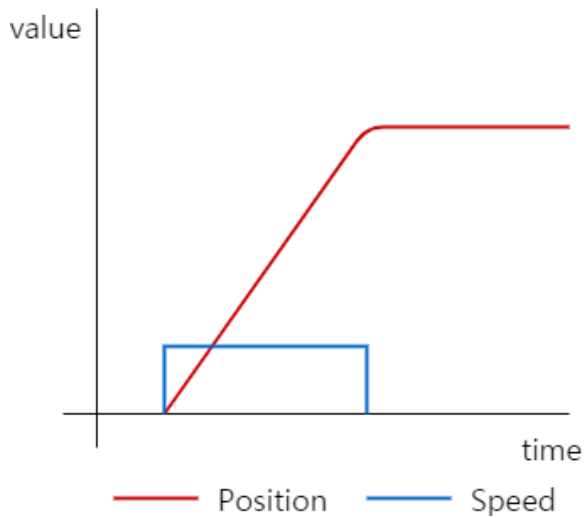


5.2 위치 제어

- 위치 제어는 절대 위치제어와 상대 위치제어로 구분된다.
 - 절대 위치제어
 - 0~65533까지 제어가 가능(0도 ~ 655.33 도)
 - 기준점(0도)을 기준으로 목표 위치로 이동
 - 예: 목표값 100도 → 항상 절대 100도 위치로 이동
 - 허용 범위보다 크게 이동하기 위해서는 감속비를 설정하여서 우회할 수 있다.
 - 상대 위치제어
 - 현재 위치를 기준으로 지정한 크기만큼 이동
 - 예: 현재 300도에서 +80도 이동이면 최종 위치 = 380도 → 360도 Overflow → 20도 위치
- 위치제어 모드는 위치제어 모드 설정을 통해서 변경된다.
- 위치 제어는 위치 이동 후 지정 위치를 유지하기 위해서 계속 제어 동작을 한다.
 - 외력에 의해서 위치가 이동될 경우 지정 위치로 이동한다.

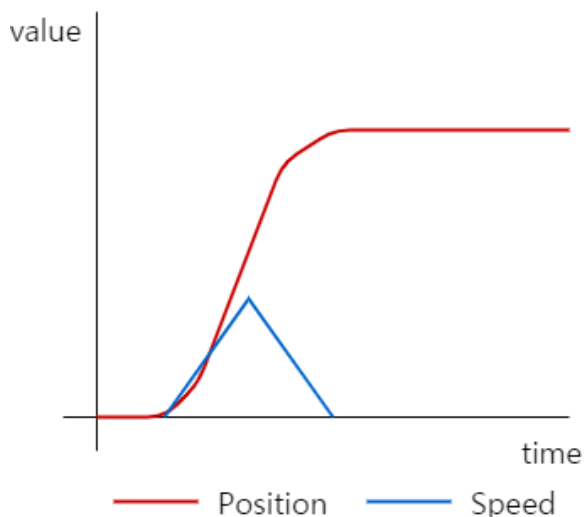
- 위치 제어는 위치제어기 설정을 통해서 PID 게인을 설정한다.
- PID는 개별 혹은 조합하여 사용이 가능하다.
- 위치제어 PID 게인 튜닝은 기구적인 외부 요인의 영향을 받으므로 항상 성능 좋은 게인 값을 찾기는 어렵다.

5.2.1 위치 속도 제어



- 현재 위치에서 목표 위치까지 지정한 속도로 이동한다.
- 방향은 0점으로 부터의 회전 방향이다.
 - 절대 위치제어
 - 고정된 0점을 기준으로 방향(CW/CCW)으로 위치값으로 이동한다.
 - 상대 위치제어
 - 현재 위치에서 지정한 방향(CW/CCW)으로 위치값만큼 이동한다.

5.2.2 가감속 위치제어



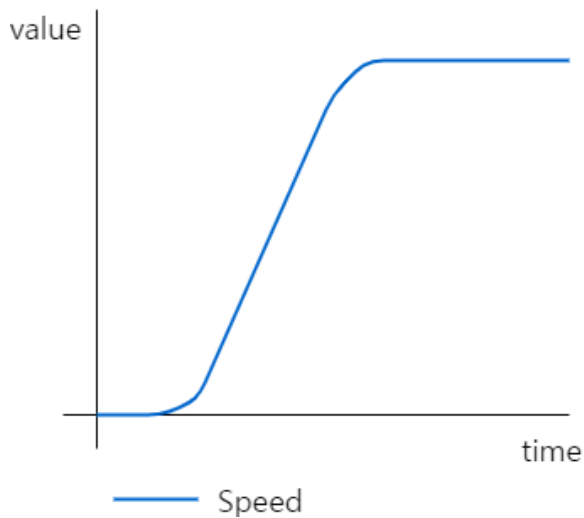
- 현재 위치에서 목표 위치까지 지정된 시간동안 이동한다.
 - 지정된 시간동안 속도는 리니어하게 가속과 감속한다.
 - 도달 시간을 2초로 설정하면, 가속 1초, 감속 1초로 동작한다.

5.3 속도 제어

- 속도 제어는 지정한 속도로 회전하도록 하는 명령이다.

- 누리로봇의 제어기는 가감속 속도 제어(accelerated speed control)만 제공한다.
- 가감속 속도제어는 모터가 목표 속도에 부드럽게 도달하도록 가속과 감속을 자동으로 제어하는 기능이다.

5.3.1 가감속 속도제어기



- 지정된 속도로 지정된 시간 동안 가속한다.
- 현재 속도가 지정된 속도보다 크면 지정된 시간동안 감속한다.

5.4 Open-loop 구동

- Closed-Loop의 다른 제어 방식과 달리 Open-Loop 구동은 피드백을 통한 보정이 되지 않는다.
 - 위치 유지 불가
- 지정된 PWM 듀티 출력을 유지한다.
 - 부하 증가 시 Stall 위험
 - 과전류 발생 가능
- 피드백에 의한 보정이 없기 때문에 상태 유지를 위한 동작이 없다.
- 각 모터별 동작하기 위한 최소 입력 전압이 다르다.
 - 같은 듀티라도 모터별로 동작 유무가 다를 수 있다.

6. 설정 명령 (Configuration Commands)

구분	Mode(hex)	BCA, DCA	SA, SB, SC	SM(구형 SC)
4	0x04	위치제어기 설정	위치제어기 설정	지원안함
5	0x05	속도제어기 설정	속도제어기 설정	속도제어기 설정
6	0x06	ID 설정	ID 설정	외부 감속비 설정
7	0x07	통신속도 설정	통신속도 설정	제어 On/Off설정
8	0x08	통신 응답시간 설정	통신 응답시간 설정	위치 초기화
9	0x09	모터 정격속도 설정	외부 감속비 설정	공장 초기화
10	0x0A	분해능 설정	제어 On/Off설정	지원안함
11	0x0B	감속비 설정	위치제어 모드 설정	지원안함
12	0x0C	제어 On/Off 설정	위치초기화	지원안함
13	0x0D	위치제어 모드 설정	공장 초기화	지원안함
14	0x0E	제어방향 설정	Echo 모드 설정	지원안함
15	0x0F	위치 초기화	지원안함	지원안함
16	0x10	공장 초기화	지원안함	지원안함

명령명	데이터 구조	기본값	비고
위치제어기 설정	[Kp, Ki, Kd, 전류]	제품별 상이	내부 제어기 게인 설정
속도제어기 설정	[Kp, Ki, Kd, 전류]	제품별 상이	
ID 설정	[New ID]	0x00	중복 금지
통신속도 설정	[Baudrate 코드]	0x06 (9,600bps)	부록 참고
통신 응답시간 설정	[시간 코드]	0x01	100 s 단위
감속비 / 외부 감속비 설정	[ratio(2B)]	10:1	제품별 차이
제어 On/Off, 위치제어 모드, 초기화	[1B]	0x00(On)	
공장 초기화			모든 설정 초기화

6.1 설정 명령 공통 유의사항

항목	설명
설정 명령 처리 시간	내부 EEPROM에 기록되므로 약 50~300ms 지연이 발생한다. 연속 명령 전송 시 최소 300ms 이상 간격이 필요하다.
피드백 확인 필요	설정 후 반드시 피드백 요청으로 변경값을 검증해야 한다.
전원/통신 안정 상태에서만 수행	설정 도중 통신 끊김 또는 전원 차단 시 데이터 손상(EEPROM Error)이 발생할 수 있다.
제어 중 설정 변경 금지	제어 루프가 활성화된 상태에서 제어기, 분해능, 감속비 등의 설정을 변경하면 오작동 또는 과전류가 발생할 수 있다.

6.2 설정 명령 설명

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	위치제어기 Kp [1byte]	위치제어기 Ki [1byte]	위치제어기 Kd [1byte]	위치제어 정격전류 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x06	부록 참고	(위치제어기 설정)	0x01 ~ 0xFE (1 ~ 254) (초기값: 제품별 상이)	0x00 ~ 0xFE (0 ~ 254) (초기값: 제품별 상이)	0x00 ~ 0xFE (0 ~ 254) (초기값: 제품별 상이)	0x01 ~ 정격전류 (1 ~ 정격전류) [100mA] (초기값: 정격전류)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	속도제어기 Kp [1byte]	속도제어기 Ki [1byte]	속도제어기 Kd [1byte]	속도제어 정격전류 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x06	부록 참고	(속도제어기 설정)	0x01 ~ 0xFE (1 ~ 254) (초기값: 제품별 상이)	0x00 ~ 0xFE (0 ~ 254) (초기값: 제품별 상이)	0x00 ~ 0xFE (0 ~ 254) (초기값: 제품별 상이)	0x01 ~ 정격전류 (1 ~ 정격전류) [100mA] (초기값: 정격전류)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	ID [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	(ID 설정)	0x00 ~ 0xFE (0 ~ 254) {초기값: 0x00}

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	통신 속도 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	(통신 속도 설정)	0x00 ~ 0x11 (0 ~ 17) [부록 참고] {초기값: 0x06}

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	통신 응답 시간 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	(통신 응답 시간 설정)	0x00 ~ 0xFE (0 ~ 254) [100us] {초기값: 0x01}

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	모터 정격 속도 [2byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x04	부록 참고	(모터 정격 속도 설정)	0x0001 ~ 0xFFFFD (1 ~ 65533) [RPM] {초기값: 0x0BB8}

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	분해능 [2byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x04	부록 참고	(분해능 설정)	0x0001 ~ 0xFFFFD (1 ~ 65533) [Encoder pulse or pole] {초기값: 0x0004}

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	감속비 [2byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x04	부록 참고	(감속비 설정)	0x0001 ~ 0xFFFFD (1 ~ 65533) [0.1ratio] {초기값: 0x000A}

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	제어 On/Off [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	(제어 On/Off 설정)	0x00 (On) 0x01 (Off) {초기값: 0x00}

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	절대/상대 위치제어 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	(위치제어 모드 설정)	0x00 (절대 위치제어) 0x01 (상대 위치제어) {초기값: 0x00}

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	제어 방향 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	(제어 방향 설정)	0x00 (CCW 방향) 0x01 (CW 방향) {초기값: 0x00}

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x02	부록 참고		(위치 초기화)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x02	부록 참고		(공장 초기화)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	Echo On/Off [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	(Echo 모드 설정)	0x00 (Echo Off) 0x01 (Echo On) {초기값: 0x00}

6.2.1 위치제어기 설정

• 기능설명

- 위치 제어기의 PID 게인(Kp, Ki, Kd) 및 정격전류(Max Current)를 설정한다.
- 제어기의 응답성, 위치 정밀도, 오버슈트 등에 직접적인 영향을 미친다.

- 유의사항
 - 잘못된 게인 설정 시 진동 및 발진 발생 가능
 - 전류 값은 모터 정격전류 이하로 설정
 - 정격전류 이상 설정 시 발열 또는 과전류 보호 작동
 - 설정 후 반드시 피드백으로 값 확인 필요

6.2.2 속도제어기 설정

- 기능설명
 - 속도 제어기의 PID 게인과 속도 제어용 정격전류를 설정한다.
 - 모터의 회전 속도 안정성 및 응답 특성을 조정할 때 사용된다.
- 유의사항
 - 잘못된 게인 설정 시 진동 및 발진 발생 가능
 - 전류 값은 모터 정격전류 이하로 설정
 - 정격전류 이상 설정 시 발열 또는 과전류 보호 작동
 - 설정 후 반드시 피드백으로 값 확인 필요

6.2.3 ID 설정

- 기능설명
 - 모듈의 고유 ID를 변경한다.
 - 각 RS485 네트워크에서 장치를 구분하는 식별 번호로 사용된다.
- 유의사항
 - ID는 0x00~0xFE 범위 내에서 중복 금지
 - Broadcast ID(0xFF)는 제어용이 아닌 공통 명령용
 - Broadcast로 명령 전달 시 동일 ID로 설정된다.
 - 시스템에서 예외처리하지 않는다.

6.2.4 통신속도 설정

- 기능설명
 - 통신 Baudrate를 설정한다.
- 유의사항
 - 제어기와 슬레이브의 Baudrate 반드시 일치해야 함
 - 설정 직후 재통신 시도 필요
 - 잘못된 설정 시 통신 불가 상태 발생 가능

6.2.5 통신 응답시간 설정

- 기능설명
 - 피드백 응답 지연시간(100 s 단위)을 설정한다.
 - 여러 모듈이 동시에 응답하는 환경에서 충돌을 방지하기 위한 지연을 부여한다.

- 유의사항
 - 응답시간이 짧으면 다중모듈 환경에서 충돌 가능
 - 응답시간이 너무 길면 통신 지연 발생
 - ID별로 적절한 간격 설정 필요
 - [[모터제어기/누리로봇 제어 프로토콜 매뉴얼#2.3 Broadcast 사용 주의사항|누리로봇 제어 프로토콜 매뉴얼]] 참고

6.2.6 모터 정격속도 설정

- 기능설명
 - 모터의 정격속도(Rated Speed, 단위: RPM)를 설정한다.
 - 내부 제어기의 기준 속도 한계를 지정한다.
- 유의사항
 - 실제 모터 사양에 맞게 설정 필요
 - 과도한 정격속도 설정 시 보호 동작 또는 손상 위험
 - 변경 후 속도 피드백으로 확인 권장

6.2.7 분해능 설정

- 기능설명
 - 엔코더 또는 홀센서의 분해능(해상도)을 설정한다.
 - Encoder 지원 제품은 엔코더 펄스를 입력한다.(PPR)
 - CPR(Counts per Revolution)과 PPR(Pulses per Revolution) 차이를 구분하라.
 - CPR : $CPR / 4$ 입력
 - PPR: PPR 입력
 - 기본 제품은 홀센서 극수를 입력한다.(Number of Poles)
 - Poles과 Pole Pair 구분하라.
 - Pole : 그대로 입력
 - Pole Pair : $Pole\ Pair \times 2$
- 유의사항
 - 현재 하드웨어 사양과 일치해야 정확한 제어 가능
 - 잘못된 설정 시 위치 제어 오차 또는 속도 이상 발생
 - 모터 모델 변경 시 반드시 재설정

6.2.8 감속비 설정

- 기능설명
 - 내부 또는 외부 감속기(Gear Ratio)를 설정한다.
 - 모터 회전량과 출력축의 각도/속도를 변환하는 비율을 지정한다.
- 유의사항

- 감속비 계산식: 출력속도 = 모터속도 ÷ 감속비
- 실제 감속비와 다르면 위치 오차 누적 발생
- 외부감속비 설정은 RSA, SA 제품에서 주로 사용

6.2.9 제어 On/Off 설정

- 기능설명
 - 모터 드라이버의 제어 상태를 켜거나 끈다. (0x00=On, 0x01=Off)
- 유의사항
 - 제어 Off 시 PWM 출력 및 제어 루프 정지
 - On 상태에서 제어기 파라미터 변경 금지

6.2.10 위치제어 모드 설정

- 기능설명
 - 절대 위치제어(0x00) 또는 상대 위치제어(0x01) 모드를 선택한다.
- 유의사항
 - 절대모드: 지정된 절대 좌표로 이동
 - 상대모드: 현재 위치를 기준으로 이동
 - 잘못 설정 시 위치 이동 방향 오류 발생 가능

6.2.11 제어방향 설정

- 기능설명
 - 모터의 회전 방향 기준을 설정한다.
- 유의사항
 - 실제 배선 방향과 불일치 시 동작 반대 발생
 - 초기 구동 시 반드시 확인 필요
 - 잘못된 방향 설정은 모터 발산의 원인

6.2.12 위치 초기화

- 기능설명
 - 현재 위치를 0도로 초기화한다.
 - 홈 포지션 지정 시 사용된다.
- 유의사항
 - 제어 Off 상태에서 수행 권장
 - 초기화 후 피드백 기준점 변경
 - 홈센서 등 외부 기준 신호와 병행 시 주의 필요

6.2.13 공장 초기화

- 기능설명

- 모든 설정값을 출하 초기값으로 복원한다.
- EEPROM에 저장된 설정이 모두 재설정된다.
- 유의사항
 - 모든 사용자 설정값 삭제됨
 - 실행 전 반드시 백업 필요 (캡처 등)
 - Broadcast(0xFF) ID로 전송 시 모든 모듈 초기화됨
 - 실행 후 재부팅 필수

6.2.14 Echo 모드 설정

- 기능설명
 - 송신 명령에 대해서 에코(반송) 설정을 하면 전송된 메시지를 확인해서 통신 무결성을 확보할 수 있음.
- 유의사항
 - 일부 제품만 제공
 - 반송 데이터에 의한 다중 제어 시 충돌 발생할 수 있음.
 - 다중 모터 제어 시 반송 데이터 수신 시간을 감안하여야 함.

7. 피드백 응답 (Feedback Response)

구분	Mode(hex)	BCA, DCA	SA, SB, SC	SM(구형 SC)
31	0xD0	Ping 응답	Ping 응답	Ping 응답
32	0xD1	위치 피드백 응답	위치 피드백 응답	속도 피드백 응답
33	0xD2	속도 피드백 응답	속도 피드백 응답	속도제어기 피드백 응답
34	0xD3	위치제어기 피드백 응답	위치제어기 피드백 응답	통신 응답시간 피드백 응답
35	0xD4	속도제어기 피드백 응답	속도제어기 피드백 응답	외부 감속비 피드백 응답
36	0xD5	통신 응답시간 피드백 응답	통신 응답시간 피드백 응답	제어 On/Off 피드백 응답
37	0xD6	모터 정격속도 피드백 응답	외부 감속비 피드백 응답	펌웨어 버전 피드백 응답
38	0xD7	분해능 피드백 응답	제어 On/Off 피드백 응답	지원안함
39	0xD8	감속비 피드백 응답	위치제어 모드 피드백 응답	지원안함
40	0xD9	제어 On/Off 피드백 응답	절대엔코더 피드백 응답	지원안함
41	0xDA	위치제어 모드 피드백 응답	Echo 모드 피드백 응답	지원안함
42	0xDB	제어방향 피드백 응답	지원안함	지원안함
43	0xFD	펌웨어 버전 응답	펌웨어 버전 응답	펌웨어 버전 응답

7.1 공통 유의사항

항목	설명
요청/응답 구조	피드백 응답(0xD0~0xDB)은 반드시 요청 명령(0xA0~0xAB) 이후에만 전송됩니다.
응답 주기	10ms~50ms 간격 이내에서 요청해야 안정적이며, 폴링 속도가 과도하면 통신 과부하가 발생할 수 있습니다.
통신 충돌 방지	여러 노드가 동시에 피드백하지 않도록 각 ID의 응답시간을 차등 설정해야 합니다.
Checksum 검증	수신 데이터는 반드시 Checksum을 계산하여 검증해야 합니다. (NOT(Sum of bytes))
피드백 오류 발생 시	데이터 깨짐 또는 지연 발생 시 RS485 선로(쉴드, GND, 종단저항)를 확인해야 합니다.
Broadcast 제한	Broadcast(0xFF)로 요청하면 모든 노드가 응답하여 충돌이 발생하므로 개별 ID로 요청해야 합니다. Broadcast를 사용하려는 경우에는 통신 응답시간을 설정하여 충돌을 회피해야 합니다.

- 위치: 0.01도 단위
 - 속도 피드백 상의 위치는 0.1도 단위이다.
- 속도: 0.1RPM 단위
- 전류: 100mA 단위
- 방향: 0x00(CCW), 0x01(CW)

7.2.1 Ping 응답

- 기능설명
 - 장치가 통신 가능 상태인지 확인하는 기본 응답이다.
 - 요청에 대한 회신으로, ID 및 연결 상태를 검증할 때 사용한다.
- 유의사항
 - 통신 정상 여부 확인용으로 사용
 - 응답이 없을 경우 배선, GND, Baudrate 확인
 - Broadcast ID(0xFF)로 Ping 요청하여 응답이 충돌하면 응답을 못 받을 수 있음

7.2.2 위치 피드백 응답

- 기능설명
 - 현재 회전 방향, 위치, 속도, 전류 정보를 반환한다.
- 유의사항
 - 위치 데이터 단위는 0.01°
 - 속도 단위 0.1RPM, 전류 단위 100mA
 - 회전 방향 (0x00 = CCW, 0x01 = CW) 확인
 - 연속 요청 시 10ms 이상 간격 유지

7.2.3 속도 피드백 응답

- 기능설명
 - 현재 속도, 위치, 전류를 피드백한다.
 - 위치제어 모드가 아닌 상태에서 실시간 속도 모니터링에 사용된다.
- 유의사항
 - 단위: 속도 0.1RPM, 위치 0.1°
 - 제어 Off 상태에서도 응답 가능
 - 고속 폴링 시 CPU 부하 증가 주의

7.2.4 위치제어기 피드백 응답

- 기능설명
 - 위치제어기의 내부 PID 값(K_p , K_i , K_d) 및 정격전류를 반환한다.
- 유의사항
 - 단위: 1바이트 = 정규화된 제어 게인 값
 - 변경 후 값 확인용으로 사용
 - 제어기 설정 후 200ms 이상 대기 후 요청

7.2.5 속도제어기 피드백 응답

- 기능설명
 - 속도제어기의 PID 값(K_p , K_i , K_d)과 정격전류를 반환한다.
- 유의사항
 - 설정 값이 EEPROM에 저장되어야 반영
 - 속도제어 루프의 안정성 점검용
 - 실시간 조정 시 발산 주의

7.2.6 통신 응답시간 피드백 응답

- 기능설명
 - 장치의 통신 응답 지연시간(100 s 단위)을 반환한다.
- 유의사항
 - 통신지연 보정용
 - 여러 장치가 있을 경우 ID별 시간차 부여 확인
 - 값이 0이거나 불규칙하면 버스 충돌 가능성

7.2.7 모터 정격속도 피드백 응답

- 기능설명
 - 설정된 모터 정격속도(RPM 단위)를 반환한다.
- 유의사항
 - 단위: 1RPM
 - 실제 모터 스펙과 일치 확인
 - 잘못된 설정 시 제어기 계산 오류 발생
 - 속도 프로파일에 도달하지 못할 경우 처리되는 동안 다른 처리가 제한될 수 있음

7.2.8 분해능 피드백 응답

- 기능설명
 - 엔코더/홀센서의 분해능(pulse 또는 pole)을 반환한다.
- 유의사항
 - 단위: pulse 또는 pole 수
 - 실제 하드웨어와 불일치 시 위치제어 불안정
 - 모터 교체 시 반드시 확인 필요

7.2.9 감속비 피드백 응답

- 기능설명
 - 설정된 감속비(0.1 단위)를 반환한다.
- 유의사항
 - 단위: 0.1 ratio
 - 예: 0x0064 = 10:1 감속비
 - 외부 감속기 존재 시 반드시 설정 확인

7.2.10 제어 On/Off 피드백 응답

- 기능설명
 - 현재 제어기 On/Off 상태를 반환한다 (0x00 = On, 0x01 = Off).
- 유의사항
 - 제어 Off 상태에서는 제어 명령이 무시됨
 - On/Off 전환 후 100ms 대기 필요
 - EEPROM 기록 중에는 On 금지

7.2.11 위치제어 모드 피드백 응답

- 기능설명
 - 현재 위치제어 모드가 **절대(0x00)** 인지 **상대(0x01)** 인지를 반환한다.
- 유의사항
 - 절대모드: 고정 기준좌표 이동
 - 상대모드: 현재 위치 기준 이동
 - 응용프로그램에서 모드 일치 여부 반드시 확인

7.2.12 제어방향 피드백 응답

- 기능설명
 - 모터의 제어 기준 방향을 반환한다 (0x00 = CCW, 0x01 = CW).
- 유의사항
 - 제어방향 설정과 피드백 일치 여부 확인
 - 잘못된 방향 설정 시 위치/속도 반전
 - 배선 변경 시 반드시 확인

7.2.13 펌웨어 버전 응답

- 기능설명
 - 제어기의 펌웨어 버전 정보를 반환한다.
- 유의사항
 - 버전별 기능 차이 가능
 - 펌웨어 업데이트 전/후 비교용
 - 버전 불일치 시 명령어 호환성 주의

7.2.14 절대엔코더 피드백 응답

- 기능설명
 - 절대엔코더가 추가된 제품의 경우 엔코더를 이용하여 모터의 절대 위치를 반환한다.
- 유의사항
 - 절대엔코더가 포함된 제품에서만 제공
 - 엔코더 옵션이 A 인 제품에서만 지원

7.2.15 Echo 모드 피드백 응답

- 기능설명
 - Echo(반송) 설정값을 반환한다.
- 유의사항
 - 0x00 : Echo 사용 안 함
 - 0x01 : Echo 사용

8. 통신 처리 로직 (Communication Logic)

8.1 송신 구조

- 제어기 → 모듈 방향으로 데이터 전송
- 파라미터를 구성하여 패킷 전송
- Checksum 계산 및 Header 포함 필수

8.2 수신 파싱 알고리즘

1. Header (0xFFFE) 탐색
2. Data Size 기준으로 프레임 완성
3. Checksum 검증
4. Mode 값 해석 후 명령별 데이터 파싱

8.3 다중 데이터 처리 시 주의사항

- 수신 중 새로운 Header가 검출될 경우 **이전 패킷 폐기**
- Broadcast 명령은 응답하지 않음
- 설정 명령 수행 후 피드백 요청 시 **충분한 지연 시간 확보 필요**

9. 예제 (Examples)

9.1 Python 시리얼 제어 예제

예제 소스는 아래 저장소에서 확인할 수 있다.

- <https://github.com/ckdo8008/nurirobotPython/blob/main/testcode.py>

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Nurirobot RS-485 control demo for MC/SA series
- Read current settings via feedback requests, then re-apply the same settings via "set" commands
- Run: Pos&Speed (0x01), AccPos (0x02), AccSpeed (0x03), and (if SA) Open-Loop (0x11) for 1s each, then stop
Spec refs:
* MC-RS485 V1.0.1: control (0x01~0x03), setters (0x04~0x0F/0x10), feedback (0xA0.. -> 0xD0..FD)
* SA-RS485 V1.0.2: Open-Loop drive (0x11)
"""

import argparse
import time
from dataclasses import dataclass
from typing import List, Dict, Optional

import serial
try:
    from serial.rs485 import RS485Settings
except Exception:
    RS485Settings = None
```

프로토콜 상수와 프레임 파서:

```
# -----
# Common protocol constants
# -----
HEADER = bytes((0xFF, 0xFE))

# Mode values (MC shared)
MODE_POS_SPEED      = 0x01 # 위치, 속도제어 (direction[1], pos[2](0.01°), spd[2](0.1RPM))
MODE_ACC_POS        = 0x02 # 가감속 위치제어 (direction[1], pos[2], arrive[1](0.1s))
MODE_ACC_SPEED      = 0x03 # 가감속 속도제어 (direction[1], spd[2], arrive[1](0.1s))
MODE_SET_POS_CTRL   = 0x04 # 위치제어기 설정 (Kp,Ki,Kd, rated[0.1A])
MODE_SET_SPD_CTRL   = 0x05 # 속도제어기 설정 (Kp,Ki,Kd, rated[0.1A])
MODE_SET_ID         = 0x06 # ID 설정 (주의: 실장비 변경 위험)
MODE_SET_BAUD       = 0x07 # 통신속도 설정 (주의: 즉시 변경)
MODE_SET_RESPTIME   = 0x08 # 통신 응답시간 설정 (value[100us])
MODE_SET_RATED_SPD  = 0x09 # 모터 정격속도 설정 (RPM, 2B)
MODE_SET_RESOLUTION = 0x0A # 분해능 설정 (2B)
MODE_SET_RATIO      = 0x0B # 감속비 설정 (0.1ratio, 2B)
MODE_SET_ONOFF      = 0x0C # 제어 On/Off (0x00 On, 0x01 Off)
MODE_SET_POS_MODE    = 0x0D # 위치제어 모드 (0x00 절대, 0x01 상대)
MODE_SET_CTRL_DIR    = 0x0E # 제어 방향 (0x00 CCW, 0x01 CW)
MODE_RESET_POS      = 0x0F # 위치 초기화
MODE_RESET_FACTORY   = 0x10 # 공장 초기화 (주의)

# SA series only
MODE_OPEN_LOOP      = 0x11 # Open-Loop 구동 (direction[1], duty[2] in 0.01%)

# Feedback requests (송신)
REQ = {
    "ping": 0xA0,
```



```

    "pos":          0xA1,
    "spd":          0xA2,
    "pos_ctrl":     0xA3,
    "spd_ctrl":     0xA4,
    "resp_time":    0xA5,
    "rated_speed":  0xA6,
    "resolution":   0xA7,
    "ratio":        0xA8,
    "ctrl_onoff":   0xA9,
    "pos_ctrl_mode": 0xAA,
    "ctrl_dir":     0xAB,
    "firmware":     0xCD,
}

# Feedback responses (수신)
FEED = {
    "ping":         0xD0,
    "pos":          0xD1,
    "spd":          0xD2,
    "pos_ctrl":     0xD3,
    "spd_ctrl":     0xD4,
    "resp_time":    0xD5,
    "rated_speed":  0xD6,
    "resolution":   0xD7,
    "ratio":        0xD8,
    "ctrl_onoff":   0xD9,
    "pos_ctrl_mode": 0xDA,
    "ctrl_dir":     0xDB,
    "firmware":     0xFD,
}

# -----
# Bytes / checksum helpers
# -----
def be16(msb: int, lsb: int) -> int:
    return ((msb & 0xFF) << 8) | (lsb & 0xFF)

def to_be16(v: int) -> bytes:
    return bytes(((v >> 8) & 0xFF, v & 0xFF))

def calc_checksum(frame_with_placeholder: bytes) -> int:
    # ~(sum of all bytes except header[0:2] and checksum[4]) & 0xFF
    s = 0
    for i, b in enumerate(frame_with_placeholder):
        if i in (0, 1, 4):
            continue
        s = (s + (b & 0xFF)) & 0xFF
    return (~s) & 0xFF

def build_frame(dev_id: int, mode: int, data: bytes = b'') -> bytes:
    # Frame: FF FE | ID | LEN | CHK | MODE | DATA...
    # LEN includes CHK + MODE + DATA
    size = 1 + 1 + len(data)
    buf = bytearray(6 + len(data))
    buf[0:2] = HEADER
    buf[2] = dev_id & 0xFF
    buf[3] = size & 0xFF
    buf[5] = mode & 0xFF
    if data:
        buf[6:] = data
    buf[4] = calc_checksum(buf)

```

```

    return bytes(buf)

# -----
# Stream parser (robust)
# -----
class StreamParser:
    def __init__(self):
        self.buf = bytearray()

    def feed(self, chunk: bytes) -> List[bytes]:
        frames: List[bytes] = []
        if not chunk:
            return frames
        self.buf.extend(chunk)

        while True:
            start = self.buf.find(HEADER)
            if start < 0:
                self.buf.clear()
                break
            if start > 0:
                del self.buf[:start]
            if len(self.buf) < 6:
                break

            length = self.buf[3]
            total = length + 4
            if total < 6:
                del self.buf[:2]
                continue

            if len(self.buf) < total:
                nxt = self.buf.find(HEADER, 2)
                if 0 <= nxt < len(self.buf):
                    del self.buf[:nxt]
                    continue
                break

            frame = bytes(self.buf[:total])
            try:
                print("Recv Frame:", ' '.join(f"{b:02X}" for b in frame))
            except Exception:
                # 출력 도중 문제가 생겨도 파싱에는 영향이 없도록 무시합니다.
                pass

            if calc_checksum(frame) != frame[4]:
                del self.buf[:2]
                continue
            frames.append(frame)
            del self.buf[:total]
        return frames

```

시리얼 링크와 피드백 파싱:

```

# -----
# Serial link
# -----
@dataclass
class LinkConfig:
    port: str
    baud: int = 9600

```

```

timeout: float = 0.2
rs485_rts_enable: bool = False

class Link:
    def __init__(self, cfg: LinkConfig):
        self.cfg = cfg
        self.ser: Optional[serial.Serial] = None
        self.parser = StreamParser()

    def open(self):
        self.ser = serial.Serial()
        self.ser.port = self.cfg.port
        self.ser.baudrate = self.cfg.baud
        self.ser.bytesize = serial.EIGHTBITS
        self.ser.parity = serial.PARITY_NONE
        self.ser.stopbits = serial.STOPBITS_ONE
        self.ser.timeout = self.cfg.timeout
        self.ser.xonxoff = False
        self.ser.rtscts = False
        self.ser.dsrtdtr = False
        if self.cfg.rs485_rts_enable and RS485Settings is not None:
            self.ser.rs485_mode = RS485Settings(
                rts_level_for_tx=True, rts_level_for_rx=False,
                delay_before_tx=0.0, delay_before_rx=0.0
            )
        self.ser.open()

    def close(self):
        if self.ser and self.ser.is_open:
            self.ser.close()

    def write(self, frame: bytes):
        self.ser.write(frame)
        self.ser.flush()
        try:
            print("Send Frame:", ' '.join(f"{b:02X}" for b in frame))
        except Exception:
            # 출력 도중 문제가 생겨도 파싱에는 영향이 없도록 무시합니다.
            pass

    def read_frames(self) -> List[bytes]:
        chunk = self.ser.read(1024)
        return self.parser.feed(chunk) if chunk else []

```

피드백 요청 대기:

```

# -----
# Feedback: read current settings (MC spec)
# -----
def request_and_wait(link: Link, dev_id: int, req_mode: int, expect_feed_mode: int, wait_s=0.35):
    frm = build_frame(dev_id, req_mode, b'')
    link.write(frm)
    end = time.monotonic() + wait_s
    while time.monotonic() < end:
        for f in link.read_frames():
            if len(f) >= 6 and f[2] == (dev_id & 0xFF) and f[5] == (expect_feed_mode & 0xFF):
                return f
        time.sleep(0.005)
    return None

```

피드백 응답 파싱:

```
def parse_feed_pos_ctrl(f: bytes) -> Dict:
    d = f[6:]
    return {"kp": d[0], "ki": d[1], "kd": d[2], "rated_current_a": d[3] * 0.1}

def parse_feed_spd_ctrl(f: bytes) -> Dict:
    d = f[6:]
    return {"kp": d[0], "ki": d[1], "kd": d[2], "rated_current_a": d[3] * 0.1}
```

현재 설정 읽기:

```
def parse_feed_settings(link: Link, dev_id: int) -> Dict:
    """
    읽을 수 있는 현재 설정/상태를 모두 피드백 요청으로 수집.
    (D3/D4/D5/D6/D7/D8/D9/DA/DB/FD 등)
    """
    out: Dict = {}
    q = [
        ("pos_ctrl", REQ["pos_ctrl"], FEED["pos_ctrl"], parse_feed_pos_ctrl),
        ("spd_ctrl", REQ["spd_ctrl"], FEED["spd_ctrl"], parse_feed_spd_ctrl),
        ("resp_time", REQ["resp_time"], FEED["resp_time"], lambda f: {"resptime_100us": f[6]}),
        ("rated_speed", REQ["rated_speed"], FEED["rated_speed"], lambda f: {"rated_rpm": be16(f[6], f[7])}),
        ("resolution", REQ["resolution"], FEED["resolution"], lambda f: {"resolution": be16(f[6], f[7])}),
        ("ratio", REQ["ratio"], FEED["ratio"], lambda f: {"ratio": be16(f[6], f[7]) / 10.0}),
        ("ctrl_onoff", REQ["ctrl_onoff"], FEED["ctrl_onoff"], lambda f: {"on": (f[6] == 0x00)}),
        ("pos_ctrl_mode", REQ["pos_ctrl_mode"], FEED["pos_ctrl_mode"], lambda f: {"pos_mode": "absolute" if f[6] == 0x00 else "relative"}),
        ("ctrl_dir", REQ["ctrl_dir"], FEED["ctrl_dir"], lambda f: {"ctrl_dir": "CCW" if f[6] == 0x00 else "CW"}),
        ("firmware", REQ["firmware"], FEED["firmware"], lambda f: {"fw_version": f[6]}),
    ]
    for key, req_m, feed_m, parser in q:
        fr = request_and_wait(link, dev_id, req_m, feed_m)
        if fr:
            out[key] = parser(fr)
        else:
            out[key] = None
    return out
```

설정 및 제어 명령:

```
# -----
# Setters (MC spec + SA Open-Loop)
# -----

def set_pos_ctrl(link: Link, dev_id: int, kp: int, ki: int, kd: int, rated_current_a: float):
    data = bytes((kp & 0xFF, ki & 0xFF, kd & 0xFF, int(round(rated_current_a * 10)) & 0xFF))
    link.write(build_frame(dev_id, MODE_SET_POS_CTRL, data))

def set_spd_ctrl(link: Link, dev_id: int, kp: int, ki: int, kd: int, rated_current_a: float):
    data = bytes((kp & 0xFF, ki & 0xFF, kd & 0xFF, int(round(rated_current_a * 10)) & 0xFF))
    link.write(build_frame(dev_id, MODE_SET_SPD_CTRL, data))

def set_resp_time(link: Link, dev_id: int, hundred_us: int):
    link.write(build_frame(dev_id, MODE_SET_RESPTIME, bytes((hundred_us & 0xFF,))))

def set_rated_speed(link: Link, dev_id: int, rpm: int):
    link.write(build_frame(dev_id, MODE_SET_RATED_SPD, to_be16(max(1, min(65533, int(rpm)))))

def set_resolution(link: Link, dev_id: int, res: int):
    link.write(build_frame(dev_id, MODE_SET_RESOLUTION, to_be16(max(1, min(65533, int(res)))))

def set_ratio(link: Link, dev_id: int, ratio: float):
    raw = max(1, min(65533, int(round(ratio * 10.0))))
    link.write(build_frame(dev_id, MODE_SET_RATIO, to_be16(raw)))

def set_onoff(link: Link, dev_id: int, on: bool):
    link.write(build_frame(dev_id, MODE_SET_ONOFF, bytes((0x00 if on else 0x01,))))

def set_pos_mode(link: Link, dev_id: int, absolute: bool):
    link.write(build_frame(dev_id, MODE_SET_POS_MODE, bytes((0x00 if absolute else 0x01,))))

def set_ctrl_dir(link: Link, dev_id: int, cw: bool):
```

```

link.write(build_frame(dev_id, MODE_SET_CTRL_DIR, bytes((0x01 if cw else 0x00,))))

def reset_pos(link: Link, dev_id: int):
    link.write(build_frame(dev_id, MODE_RESET_POS, b''))

# -----
# Controllers (run then stop)
# -----

def cmd_pos_speed(link: Link, dev_id: int, direction_cw: bool, pos_deg: float, spd_rpm: float):
    pos_raw = max(0, min(65533, int(round(pos_deg * 100.0)))) # 0.01°
    spd_raw = max(0, min(65533, int(round(spd_rpm * 10.0)))) # 0.1RPM
    data = bytes((0x01 if direction_cw else 0x00,)) + to_be16(pos_raw) + to_be16(spd_raw)
    link.write(build_frame(dev_id, MODE_POS_SPEED, data))

def cmd_acc_pos(link: Link, dev_id: int, direction_cw: bool, pos_deg: float, arrive_s: float):
    pos_raw = max(0, min(65533, int(round(pos_deg * 100.0)))) # 0.01°
    arr_raw = max(1, min(255, int(round(arrive_s * 10.0)))) # 0.1s
    data = bytes((0x01 if direction_cw else 0x00,)) + to_be16(pos_raw) + bytes((arr_raw,))
    link.write(build_frame(dev_id, MODE_ACC_POS, data))

def cmd_acc_speed(link: Link, dev_id: int, direction_cw: bool, spd_rpm: float, arrive_s: float):
    spd_raw = max(0, min(65533, int(round(spd_rpm * 10.0)))) # 0.1RPM
    arr_raw = max(1, min(255, int(round(arrive_s * 10.0)))) # 0.1s
    data = bytes((0x01 if direction_cw else 0x00,)) + to_be16(spd_raw) + bytes((arr_raw,))
    link.write(build_frame(dev_id, MODE_ACC_SPEED, data))

def cmd_open_loop(link: Link, dev_id: int, direction_cw: bool, duty_percent: float):
    # SA 전용: duty in 0.01% (0..100.00%)
    duty_raw = max(0, min(10000, int(round(duty_percent * 100.0))))
    data = bytes((0x01 if direction_cw else 0x00,)) + to_be16(duty_raw)
    link.write(build_frame(dev_id, MODE_OPEN_LOOP, data))

def safe_stop(link: Link, dev_id: int):
    # 위치제어, 가감속 위치제어에서 사용하면 위치가 초기화됨
    # 스톱 등의 위험상황에서 사용하는 코드
    # 속도를 0RPM으로 짧게 감속 후 제어 Off
    cmd_acc_speed(link, dev_id, direction_cw=False, spd_rpm=0.0, arrive_s=0.2)
    time.sleep(0.25)
    set_onoff(link, dev_id, on=False) # 0x01 = Off
    time.sleep(0.05)

```

데모 실행 흐름:

```

# -----
# Demo flow
# -----

def demo(series: str, port: str, dev_id: int, baud: int, rs485_rts_enable: bool):
    link = Link(LinkConfig(port=port, baud=baud, rs485_rts_enable=rs485_rts_enable))
    link.open()
    try:
        print(f"[OPEN] {port} @ {baud}bps, id=0x{dev_id:02X}, series={series}")

        # 1) 현재 설정값 읽기 (feedback)
        cur = parse_feed_settings(link, dev_id)
        print("\n[CURRENT SETTINGS]")
        for k, v in cur.items():
            print(f"- {k}: {v}")

        # 2) 읽은 설정을 그대로 설정(송신)으로 반영 (안전 데모)
        if cur.get("pos_ctrl"):
            set_pos_ctrl(link, dev_id, **cur["pos_ctrl"])

```

```

if cur.get("spd_ctrl"):
    set_spd_ctrl(link, dev_id, **cur["spd_ctrl"])
if cur.get("resp_time"):
    set_resp_time(link, dev_id, cur["resp_time"]["resptime_100us"])
if cur.get("rated_speed"):
    set_rated_speed(link, dev_id, cur["rated_speed"]["rated_rpm"])
if cur.get("resolution"):
    set_resolution(link, dev_id, cur["resolution"]["resolution"])
if cur.get("ratio"):
    set_ratio(link, dev_id, cur["ratio"]["ratio"])
if cur.get("pos_ctrl_mode"):
    set_pos_mode(link, dev_id, cur["pos_ctrl_mode"]["pos_mode"] = "absolute")
if cur.get("ctrl_dir"):
    set_ctrl_dir(link, dev_id, cw=(cur["ctrl_dir"]["ctrl_dir"] == "CW"))
if cur.get("ctrl_onoff"):
    set_onoff(link, dev_id, on=(cur["ctrl_onoff"]["on"]))

time.sleep(0.05)

# 3) 제어 데모: 각 1초 구동 후 정지
print("\n[DEMO] 제어 On")
set_onoff(link, dev_id, on=True)          # 제어 On (0x00)
time.sleep(0.05)

# (a) 위치, 속도제어 1s
print("[RUN] 위치, 속도제어 (0x01) -> 1s")
# 예: CCW로 180.00도, 5.0RPM
cmd_pos_speed(link, dev_id, direction_cw=False, pos_deg=180.00, spd_rpm=5.0) # MC 스펙: dir/pos(0.01°)/spd(0.1RPM):contentReference[oaicite:3]{index=3}
time.sleep(1.0)
safe_stop(link, dev_id)
print("[STOP] done\n")
time.sleep(0.2)
set_onoff(link, dev_id, on=True); time.sleep(0.05)

# (b) 가감속 위치제어 1s
print("[RUN] 가감속 위치제어 (0x02) -> 1s")
# 예: CW로 360.00도, 도달시간 1.0s
cmd_acc_pos(link, dev_id, direction_cw=True, pos_deg=360.00, arrive_s=1.0) # MC 스펙: dir/pos/arrive(0.1s):contentReference[oaicite:4]{index=4}
time.sleep(1.0)
safe_stop(link, dev_id)
print("[STOP] done\n")
time.sleep(0.2)
set_onoff(link, dev_id, on=True); time.sleep(0.05)

# (c) 가감속 속도제어 1s
print("[RUN] 가감속 속도제어 (0x03) -> 1s")
# 예: CCW로 100RPM, 도달시간 0.5s
cmd_acc_speed(link, dev_id, direction_cw=False, spd_rpm=100.0, arrive_s=0.5) # MC 스펙: dir/spd/arrive(0.1s):contentReference[oaicite:5]{index=5}
time.sleep(1.0)
safe_stop(link, dev_id)
print("[STOP] done\n")

set_onoff(link, dev_id, on=True); time.sleep(0.05)
print("[RUN] Open-Loop (0x11) -> 1s")
# 예: CCW 50% duty
cmd_open_loop(link, dev_id, direction_cw=False, duty_percent=50.0) # SA 스펙: dir/duty(0.01%):contentReference[oaicite:6]{index=6}
time.sleep(1.0)
safe_stop(link, dev_id)
print("[STOP] done\n")

print("[DEMO] 완료")

finally:
    link.close()

```

진입점:

```

def main():
    ap = argparse.ArgumentParser(description="Nurirobot RS-485 MC/SA control demo")
    ap.add_argument("--series", choices=["MC", "SA"], default="MC", help="제품 시리즈 (MC 기본, SA는 Open-Loop 지원)")
    ap.add_argument("--port", default="COM10", help="시리얼 포트 (예: /dev/ttyUSB0, COM5)")
    ap.add_argument("--id", type=lambda x: int(x, 0), default=0x00, help="장치 ID (기본 0x00)")
    ap.add_argument("--baud", type=int, default=115200, help="Baudrate (기본 9600)")
    ap.add_argument("--rts485", action="store_true", help="어댑터가 RTS 기반 DE/RE 전환 필요 시 사용")
    args = ap.parse_args()

    demo(args.series, args.port, args.id, args.baud, args.rts485)

if __name__ == "__main__":
    main()

```

9.2 C 시리얼 제어 예제

예제 소스는 아래 저장소에서 확인할 수 있다.

- <https://github.com/ckdo8008/nurirobotPython/blob/main/test.c>

```

#include <windows.h>
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include <chrono>
#include <thread>

using namespace std;

// -----
// Protocol constants
// -----
static const uint8_t HEADER[2] = { 0xFF, 0xFE };

// MC-control
#define MODE_POS_SPEED      0x01
#define MODE_ACC_POS       0x02
#define MODE_ACC_SPEED     0x03
#define MODE_OPEN_LOOP     0x11    // SA 전용

// Settings
#define MODE_SET_POS_CTRL   0x04
#define MODE_SET_SPD_CTRL   0x05
#define MODE_SET_ID        0x06
#define MODE_SET_BAUD      0x07
#define MODE_SET_RESPTIME  0x08
#define MODE_SET_RATEDSPD  0x09
#define MODE_SET_RESOLUTION 0x0A
#define MODE_SET_RATIO     0x0B
#define MODE_SET_ONOFF     0x0C
#define MODE_SET_POSMODE   0x0D
#define MODE_SET_CTRLDIR   0x0E
#define MODE_RESET_POS     0x0F
#define MODE_RESET_FACTORY 0x10

// Feedback Requests
#define REQ_PING            0xA0
#define REQ_POS            0xA1
#define REQ_SPD            0xA2
#define REQ_POS_CTRL       0xA3
#define REQ_SPD_CTRL       0xA4
#define REQ_RESP_TIME      0xA5
#define REQ_RATEDSPD       0xA6
#define REQ_RESOLUTION     0xA7
#define REQ_RATIO          0xA8
#define REQ_ONOFF          0xA9
#define REQ_POSMODE        0xAA
#define REQ_CTRLDIR        0xAB
#define REQ_FIRMWARE       0xCD

```

유틸리티 함수:

```

// =====
// Utility
// =====
static uint16_t be16(uint8_t hi, uint8_t lo)
{
    return ((uint16_t)hi << 8) | lo;
}

// 체크섬: 헤더(2바이트) & checksum 바이트 제외한 모든 바이트 합에 NOT

```

```

uint8_t calcChecksum(const vector<uint8_t>& frame)
{
    uint32_t sum = 0;
    for (size_t i = 0; i < frame.size(); i++)
    {
        if (i == 0 || i == 1 || i == 4) continue;
        sum += frame[i];
    }
    return (uint8_t)(~(sum & 0xFF));
}

// 프레임 생성
vector<uint8_t> buildFrame(uint8_t id, uint8_t mode, const vector<uint8_t>& data)
{
    size_t size = 1 + 1 + data.size();
    vector<uint8_t> frame(6 + data.size());

    frame[0] = 0xFF;
    frame[1] = 0xFE;
    frame[2] = id;
    frame[3] = (uint8_t)size;
    frame[5] = mode;

    if (!data.empty())
        memcpy(&frame[6], &data[0], data.size());

    frame[4] = calcChecksum(frame);
    return frame;
}

// 프레임 출력
void printFrame(const string& tag, const vector<uint8_t>& f)
{
    cout << tag;
    for (auto b : f)
        cout << " " << hex << setw(2) << setfill('0') << (int)b;
    cout << dec << endl;
}

```

시리얼 클래스와 스트림 파서:

```

// =====
// Windows Serial Class
// =====
class SerialPort
{
private:
    HANDLE hSerial = INVALID_HANDLE_VALUE;

public:
    bool openPort(const string& port, uint32_t baud)
    {
        hSerial = CreateFileA(
            port.c_str(),
            GENERIC_READ | GENERIC_WRITE,
            0,
            NULL,
            OPEN_EXISTING,
            0,
            NULL
        );
    }
}

```



```

    if (hSerial == INVALID_HANDLE_VALUE)
    {
        cout << "Failed to open port" << endl;
        return false;
    }

    DCB dcbSerialParams = { 0 };
    dcbSerialParams.DCBlength = sizeof(dcbSerialParams);

    if (!GetCommState(hSerial, &dcbSerialParams))
        return false;

    dcbSerialParams.BaudRate = baud;
    dcbSerialParams.ByteSize = 8;
    dcbSerialParams.StopBits = ONESTOPBIT;
    dcbSerialParams.Parity = NOPARITY;

    SetCommState(hSerial, &dcbSerialParams);
    return true;
}

void closePort()
{
    if (hSerial != INVALID_HANDLE_VALUE)
        CloseHandle(hSerial);
}

bool writeBytes(const vector<uint8_t>& data)
{
    DWORD written;
    BOOL ok = WriteFile(hSerial, data.data(), data.size(), &written, NULL);

    return ok && written == data.size();
}

vector<uint8_t> readBytes()
{
    uint8_t buf[1024];
    DWORD dwRead = 0;
    ReadFile(hSerial, buf, 1024, &dwRead, NULL);

    vector<uint8_t> out;
    for (size_t i = 0; i < dwRead; i++)
        out.push_back(buf[i]);

    return out;
}

};

// =====
// StreamParser (FF FE + LEN+4 기준 파싱) :contentReference[oaicite:2]{index=2}
// =====
class StreamParser
{
private:
    vector<uint8_t> buf;

public:
    vector<vector<uint8_t>> feed(const vector<uint8_t>& chunk)
    {

```

```

vector<vector<uint8_t>> frames;
buf.insert(buf.end(), chunk.begin(), chunk.end());

while (true)
{
    // FF FE 검색
    size_t pos = string::npos;
    for (size_t i = 0; i + 1 < buf.size(); i++)
    {
        if (buf[i] == 0xFF && buf[i + 1] == 0xFE)
        {
            pos = i;
            break;
        }
    }

    if (pos == string::npos)
    {
        if (buf.size() > 1)
            buf.erase(buf.begin(), buf.end() - 1);
        break;
    }

    if (pos > 0)
        buf.erase(buf.begin(), buf.begin() + pos);

    if (buf.size() < 6)
        break;

    uint8_t len = buf[3];
    size_t total = len + 4;
    if (buf.size() < total)
        break;

    vector<uint8_t> frame(buf.begin(), buf.begin() + total);

    // 체크섬 확인
    if (calcChecksum(frame) == frame[4])
        frames.push_back(frame);

    buf.erase(buf.begin(), buf.begin() + total);
}

return frames;
}
};

```

제어 명령 생성:

```

// =====
// Build control commands
// =====
vector<uint8_t> cmd_pos_speed(uint8_t id, bool cw, float pos_deg, float spd)
{
    uint16_t pos_raw = (uint16_t)(pos_deg * 100.0f); // 0.01°
    uint16_t spd_raw = (uint16_t)(spd * 10.0f); // 0.1RPM

    vector<uint8_t> d;
    d.push_back(cw ? 0x01 : 0x00);
    d.push_back((pos_raw >> 8) & 0xFF);
    d.push_back(pos_raw & 0xFF);
    d.push_back((spd_raw >> 8) & 0xFF);
}

```

```

    d.push_back(spd_raw & 0xFF);

    return buildFrame(id, MODE_POS_SPEED, d);
}

vector<uint8_t> cmd_acc_pos(uint8_t id, bool cw, float pos_deg, float arrive_s)
{
    uint16_t pos_raw = (uint16_t)(pos_deg * 100.0f);
    uint8_t arr_raw = (uint8_t)(arrive_s * 10.0f);

    vector<uint8_t> d;
    d.push_back(cw ? 1 : 0);
    d.push_back((pos_raw >> 8) & 0xFF);
    d.push_back(pos_raw & 0xFF);
    d.push_back(arr_raw);

    return buildFrame(id, MODE_ACC_POS, d);
}

vector<uint8_t> cmd_acc_speed(uint8_t id, bool cw, float spd, float arrive_s)
{
    uint16_t spd_raw = (uint16_t)(spd * 10.0f);
    uint8_t arr_raw = (uint8_t)(arrive_s * 10.0f);

    vector<uint8_t> d;
    d.push_back(cw ? 1 : 0);
    d.push_back((spd_raw >> 8) & 0xFF);
    d.push_back(spd_raw & 0xFF);
    d.push_back(arr_raw);

    return buildFrame(id, MODE_ACC_SPEED, d);
}

vector<uint8_t> cmd_open_loop(uint8_t id, bool cw, float duty_percent)
{
    uint16_t duty = (uint16_t)(duty_percent * 100);

    vector<uint8_t> d;
    d.push_back(cw ? 1 : 0);
    d.push_back((duty >> 8) & 0xFF);
    d.push_back(duty & 0xFF);

    return buildFrame(id, MODE_OPEN_LOOP, d);
}

// 위치제어, 가감속 위치제어에서 사용하면 위치가 초기화됨
// 스톱 등의 위험상황에서 사용하는 코드
// 안전 정지
void safe_stop(SerialPort& sp, uint8_t id)
{
    auto f = cmd_acc_speed(id, false, 0.0f, 0.2f);
    sp.writeBytes(f);
    this_thread::sleep_for(chrono::milliseconds(250));

    vector<uint8_t> off = buildFrame(id, MODE_SET_ONOFF, { 0x01 }); // Off
    sp.writeBytes(off);
}

```

메인 데모:

```

// =====
// Main Demo (파이썬 demo() 동일 로직)

```

```
// =====
int main()
{
    SerialPort serial;
    if (!serial.openPort("\\\\.\\COM10", 115200))
    {
        cout << "Port open failed" << endl;
        return 0;
    }

    cout << "[OPEN] COM10 115200bps" << endl;

    uint8_t id = 0x00;

    // 제어 ON
    serial.writeBytes(buildFrame(id, MODE_SET_ONOFF, { 0x00 }));
    this_thread::sleep_for(chrono::milliseconds(50));

    // ---- 위치 속도제어 ----
    cout << "[RUN] PosSpeed 1s" << endl;
    serial.writeBytes(cmd_pos_speed(id, false, 180.0f, 5.0f));
    this_thread::sleep_for(chrono::seconds(1));
    safe_stop(serial, id);

    // ---- 가감속 위치제어 ----
    serial.writeBytes(buildFrame(id, MODE_SET_ONOFF, { 0x00 }));
    cout << "[RUN] AccPos 1s" << endl;
    serial.writeBytes(cmd_acc_pos(id, true, 360.0f, 1.0f));
    this_thread::sleep_for(chrono::seconds(1));
    safe_stop(serial, id);

    // ---- 가감속 속도제어 ----
    serial.writeBytes(buildFrame(id, MODE_SET_ONOFF, { 0x00 }));
    cout << "[RUN] AccSpeed 1s" << endl;
    serial.writeBytes(cmd_acc_speed(id, false, 100.0f, 0.5f));
    this_thread::sleep_for(chrono::seconds(1));
    safe_stop(serial, id);

    // ---- Open-loop ----
    serial.writeBytes(buildFrame(id, MODE_SET_ONOFF, { 0x00 }));
    cout << "[RUN] OpenLoop 1s" << endl;
    serial.writeBytes(cmd_open_loop(id, false, 50.0f));
    this_thread::sleep_for(chrono::seconds(1));
    safe_stop(serial, id);

    serial.closePort();
    cout << "[DONE]" << endl;

    return 0;
}
```

10. 통신 오류 처리 (Error Handling)

- 헤더 불일치 → 버퍼 초기화
- Checksum 불일치 → 프레임 폐기
- 수신 타임아웃 → 재요청
- 응답 지연 발생 시 → 200~500 s 대기 후 재시도

10. 분해능 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	분해능 [2byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x04	부록 참고	0x0A (분해능 설정)	0x0001 ~ 0xFFFFD (1 ~ 65533) [Encoder pulse or pole] (조기값: 0x0004)

MC 예제: ID0을 엔코더 분해능 13pulse로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x04 0xE4 0x0A 0x00 0x0D
 BLMC-H 예제: ID0을 홀센서 분해능 8pole로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x04 0xE9 0x0A 0x00 0x08

11. 감속비 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	감속비 [2byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x04	부록 참고	0x0B (감속비 설정)	0x0001 ~ 0xFFFFD (1 ~ 65533) [0.1ratio] (조기값: 0x000A)
예제: ID0을 감속비 10:1로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x04 0x08 0x0B 0x00 0x00 0x64					

12. 제어 On/Off 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	제어 On/Off [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	0x0C (제어 On/Off 설정)	0x00 (Off) 0x01 (On) {초기값: 0x00}
예제: ID0을 제어 오프로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x03 0xEF 0x0C 0x01					

13. 위치제어 모드 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data	
			Check sum [1byte]	Mode [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	0x0D (위치제어 모드 설정)
예제: ID를 상대 위치제어 모드로 설정하기 위한 프로토콜			0xFF 0xFE 0x00 0x03 0xEE 0x0D 0x01	0x00 (절대 위치제어) 0x01 (상대 위치제어) (초기값: 0x00)

14. 제어 방향 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	제어 방향 [1byte]
0xFFFF	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	0x0E (제어 방향 설정)	0x00 (CCW 방향) 0x01 (CW 방향) (초기값: 0x00)
예제: ID의 제어 방향을 CW 방향으로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x03 0xFE 0x00 0x01					

15. 위치 초기화 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data	
			Check sum [1byte]	Mode [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x02	부록 참고	0x0F (위치 초기화)

예제: ID를 위치 초기화하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x02 0xEF 0x0E

16. 공장 초기화 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data	
			Check sum [1byte]	Mode [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x02	부록 참고	0x10 (공강 초기화)

예제: Broadcast ID로 공강 초기화하기 위한 프로토콜 - 0xFF 0xFE 0xFF 0x02 0xF1 0xD0

17. Open-Loop 구동 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data			
			Check sum [1byte]	Mode [1byte]	구동방향 [2byte]	PWM 듀티비 [2byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x05	부록 참고	0x11 (Open-Loop 구동)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0x2710 (0 ~ 10000) [0.01%

예제: ID0을 50% CCW 구동하기 위한 프로토콜 0xFF 0xFE 0x00 0x05 0x4E 0x11 0x00 0x13 0x88

17. 피드백 요청 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data	
			Check sum [1byte]	Mode [1byte]
0xFFFF	0x00 ~ 0xFE (0 ~ 254)	0x02	부록 참고	0xA0 (Ping) 0xA1 (위치 피드백) 0xA2 (속도 피드백) 0xA3 (위치제어기 피드백) 0xA4 (속도제어기 피드백) 0xA5 (통신 응답시간 피드백) 0xA6 (모터 정격속도 피드백) 0xA7 (분해능 피드백) 0xA8 (감속비 피드백) 0xA9 (제어 On/Off 피드백) 0xAA (위치제어 모드 피드백) 0xAB (제어 방향 피드백) 0xCD (펌웨어 버전 피드백)
예제: ID0을 위치 피드백하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x02 0x5C 0xA1				

18. Ping (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data	
			Check sum [1byte]	Mode [1byte]
0xFFFF	0x00 ~ 0xFE (0 ~ 254)	0x02	부록 참고	0xD0 (Ping)
예제: 0xFF 0xFE 0x00 0x02 0x2D 0xD0 - ID0의 Ping				

19. 위치 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	위치방향 [1byte]	위치 [2byte]	속도 [2byte]	전류 [1byte]
0xFFFF	0x00 ~ 0xFE (0 ~ 254)	0x08	부록 참고	0xD1 (위치 피드백)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0xFFFD (0 ~ 65533) [0.01Degree]	0x0000 ~ 0xFFFD (0 ~ 65533) [0.1RPM]	0x00 ~ 0xFE (0 ~ 254) [100mA]
예제: 0xFF 0xFE 0x00 0x08 0x90 0xD1 0x00 0x46 0x50 0x00 0x00 0x00 - ID0은 CCW방향 위치 180.00도, 속도 0.0RPM, 전류 0.0A								

20. 속도 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	속도방향 [1byte]	속도 [2byte]	위치 [2byte]	전류 [1byte]
0xFFFF	0x00 ~ 0xFE (0 ~ 254)	0x08	부록 참고	0xD2 (속도 피드백)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0xFFFD (0 ~ 65533) [0.1RPM]	0x0000 ~ 0xFFFD (0 ~ 65533) [0.1Degree]	0x00 ~ 0xFE (0 ~ 254) [100mA]
예제: 0xFF 0xFE 0x00 0x08 0x01 0xD2 0x00 0x03 0xB3 0x07 0x67 0x00 - ID0은 CCW방향 속도 94.7RPM, 위치 189.5도, 전류 0.0A								

21. 위치제어기 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	위치제어기 Kp [1byte]	위치제어기 Ki [1byte]	위치제어기 Kd [1byte]	위치제어 정격전류 [1byte]
0xFFFF	0x00 ~ 0xFE (0 ~ 254)	0x06	부록 참고	0xD3 (위치제어기 피드백)	0x01 ~ 0xFE (1 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x01 ~ 정격전류 (1 ~ 정격전류) [100mA] {초기값: 정격전류}
예제: 0xFF 0xFE 0x00 0x06 0xDC 0xD3 0xFE 0xFE 0xFE 0x50 - ID0은 위치제어기 Kp=244, Ki=244, Kd=244, 위치제어 정격전류 8.0A								

22. 속도제어기 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	속도제어기 Kp [1byte]	속도제어기 Ki [1byte]	속도제어기 Kd [1byte]	속도제어 정격전류 [1byte]
0xFFFF	0x00 ~ 0xFE (0 ~ 254)	0x06	부록 참고	0xD4 (속도제어기 피드백)	0x01 ~ 0xFE (1 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x01 ~ 정격전류 (1 ~ 정격전류) [100mA] {초기값: 정격전류}
예제: 0xFF 0xFE 0x00 0x06 0xDB 0xD4 0xFE 0xFE 0xFE 0x50 - ID0은 속도제어기 Kp=244, Ki=244, Kd=244, 속도제어 정격전류 8.0A								

23. 통신 응답시간 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	통신 응답시간 [1byte]
0xFFFF	0x00 ~ 0xFE (0 ~ 254)	0x03	부록 참고	0xD5 (통신 응답시간 피드백)	0x00 ~ 0xFE (0 ~ 254) [100us] {초기값: 0x01}
예제: 0xFF 0xFE 0x00 0x03 0x26 0xD5 0x01 - ID0의 통신 응답시간은 100us					

24. 모터 정격속도 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	모터 정격속도 [2byte]
0xFFFF	0x00 ~ 0xFE (0 ~ 254)	0x04	부록 참고	0xD6 (모터 정격속도 피드백)	0x0001 ~ 0xFFFD (1 ~ 65533) [RPM] {초기값: 0x0BB8}
예제: 0xFF 0xFE 0x00 0x04 0x62 0xD6 0x0B 0xB8 - ID0의 모터 정격속도는 3000RPM					

25. 분해능 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	분해능 [2byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x04	부록 참고	0xD7 (분해능 피드백)	0x0001 ~ 0xFFFFD (1 ~ 65533) [Encoder pulse or pole] {초기값: 0x0004}
MC 예제: 0xFF 0xFE 0x00 0x04 0x11 0xD7 0x00 0x13 - ID0의 엔코더 분해능은 19pulse					
BLMC-H 예제: 0xFF 0xFE 0x00 0x04 0x20 0xD7 0x00 0x04 - ID0의 홀센서 분해능은 4pole					

26. 감속비 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	감속비 [2byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x04	부록 참고	0xD8 (감속비 피드백)	0x0001 ~ 0xFFFFD (1 ~ 65533) [0.1ratio] {초기값: 0x000A}
예제: 0xFF 0xFE 0x00 0x04 0xBF 0xD8 0x00 0x64 - ID0의 감속비는 10:1					

27. 제어 On/Off 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	제어 On/Off [1byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x03	부록 참고	0xD9 (제어 On/Off 피드백)	0x00 (On) 0x01 (Off) {초기값: 0x00}
예제: 0xFF 0xFE 0x00 0x03 0x23 0xD9 0x00 - ID0은 제어 On					

28. 위치제어 모드 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	절대/상대 위치제어 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	0xDA (위치제어 모드 피드백)	0x00 (절대 위치제어) 0x01 (상대 위치제어) {초기값: 0x00}
예제: 0xFF 0xFE 0x00 0x03 0x22 0xDA 0x00 - ID0은 절대 위치제어 모드					

29. 제어 방향 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	제어 방향 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	0xDB (제어 방향 피드백)	0x00 (CCW 방향) 0x01 (CW 방향) {초기값: 0x00}
예제: 0xFF 0xFE 0x00 0x03 0x21 0xDB 0x00 - ID0의 제어 방향은 CCW 방향					

30. 펌웨어 버전 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	버전 [1byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x03	부록 참고	0xFD (펌웨어 버전 피드백)	0x00 ~ 0xFE (0 ~ 255)
예제: 0xFF 0xFE 0x00 0x03 0xFF 0xFD 0x00 - ID0은 펌웨어 버전 0					

스마트모터 프로토콜

누리로봇 통신프로토콜 SA-RS485_V1.0.2

1. 위치, 속도제어 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data				
			Check sum [1byte]	Mode [1byte]	위치방향 [1byte]	위치 [2byte]	속도 [2byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x07	부록 참고	0x01 (위치, 속도제어)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0xFFFF (0 ~ 65533) [0.01Degree]	0x0001 ~ 0xFFFF (1 ~ 65533) [0.1RPM]

예제: ID0을 5.0RPM의 속도로 CCW방향 180.00도에 도달하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x07 0x2F 0x01 0x00 0x46 0x50 0x00 0x32

2. 가감속 위치제어 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data				
			Check sum [1byte]	Mode [1byte]	위치방향 [1byte]	위치 [2byte]	위치 도달 시간 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x06	부록 참고	0x02 (가감속 위치제어)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0xFFFF (0 ~ 65533) [0.01Degree]	0x01 ~ 0xFF (1 ~ 255) [0.1s]

예제: ID0을 위치 도달 시간 5.0s로 CW방향 360.00도에 도달하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x06 0x98 0x02 0x01 0x8C 0xA0 0x32

3. 가감속 속도제어 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data				
			Check sum [1byte]	Mode [1byte]	속도방향 [1byte]	속도 [2byte]	속도 도달 시간 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x06	부록 참고	0x03 (가감속 속도제어)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0xFFFF (0 ~ 65533) [0.1RPM]	0x01 ~ 0xFF (1 ~ 255) [0.1s]

예제: ID0을 속도 도달 시간 1.0s로 CCW방향 10RPM에 도달하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x06 0x88 0x03 0x00 0x00 0x64 0x0A

4. 위치제어기 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	위치제어기 Kp [1byte]	위치제어기 Ki [1byte]	위치제어기 Kd [1byte]	위치제어 정격전류 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x06	부록 참고	0x04 (위치제어기 설정)	0x01 ~ 0xFE (1 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x01 ~ 정격전류 (1 ~ 정격전류) [100mA] {초기값: 정격전류}

예제: ID0을 위치제어기 Kp=244, Ki=244, Kd=0 위치제어 정격전류 3.2A로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x06 0xD9 0x04 0xFE 0xFE 0x00 0x20

5. 속도제어기 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	속도제어기 Kp [1byte]	속도제어기 Ki [1byte]	속도제어기 Kd [1byte]	속도제어 정격전류 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x06	부록 참고	0x05 (속도제어기 설정)	0x01 ~ 0xFE (1 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x01 ~ 정격전류 (1 ~ 정격전류) [100mA] {초기값: 정격전류}

예제: ID0을 속도제어기 Kp=244, Ki=244, Kd=0 속도제어 정격전류 3.2A로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x06 0xD8 0x05 0xFE 0xFE 0x00 0x20

6. ID 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	ID [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	0x06 (ID 설정)	0x00 ~ 0xFE (0 ~ 254) {초기값: 0x00}

예제: ID0을 ID1로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x03 0xF5 0x06 0x01

7. 통신속도 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	통신속도 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	0x07 (통신속도 설정)	0x00 ~ 0x11 (0 ~ 17) [부록 참고] {초기값: 0x06}

예제: ID0을 통신속도 115,200bps로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x03 0xE8 0x07 0x0D

8. 통신 응답시간 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	통신 응답시간 [1byte]
0xFFFE	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	0x08 (통신 응답시간 설정)	0x00 ~ 0xFE (0 ~ 254) [100us] {초기값: 0x01}

예제: ID0을 통신 응답시간 200us로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x03 0xF2 0x08 0x02

9. 외부 감속비 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	감속비 [2byte]
0xFFFF	0x00 ~ 0xFF (0 ~ 255)	0x04	부록 참고	0x09 (외부 감속비 설정)	0x0001 ~ 0xFFFFD (1 ~ 65533) [0.1ratio] {초기값: 0x000A}

예제: ID0을 외부 감속비 2:1로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x04 0xDE 0x09 0x00 0x14

10. 제어 On/Off 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	제어 On/Off [1byte]
0xFFFF	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	0x0A (제어 On/Off 설정)	0x00 (On) 0x01 (Off) {초기값: 0x00}

예제: ID0을 제어 Off로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x03 0xF1 0x0A 0x01

11. 위치제어 모드 설정 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	절대/상대 위치제어 [1byte]
0xFFFF	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	0x0B (위치제어 모드 설정)	0x00 (절대 위치제어) 0x01 (상대 위치제어) {초기값: 0x00}

예제: ID0을 상대 위치제어 모드로 설정하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x03 0xF0 0x0B 0x01

12. 위치 초기화 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data	
			Check sum [1byte]	Mode [1byte]
0xFFFF	0x00 ~ 0xFF (0 ~ 255)	0x02	부록 참고	0x0C (위치 초기화)

예제: ID0을 위치 초기화하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x02 0xF1 0x0C

13. 공장 초기화 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data	
			Check sum [1byte]	Mode [1byte]
0xFFFF	0x00 ~ 0xFF (0 ~ 255)	0x02	부록 참고	0x0D (공장 초기화)

예제: Broadcast ID로 공장 초기화하기 위한 프로토콜 - 0xFF 0xFE 0xFF 0x02 0xF1 0x0D

14. Open-Loop 구동 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data			
			Check sum [1byte]	Mode [1byte]	구동방향 [1byte]	PWM 듀티비 [2byte]
0xFFFF	0x00 ~ 0xFF (0 ~ 255)	0x05	부록 참고	0x11 (Open-Loop 구동)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0x2710 (0 ~ 10000) [0.01%]

예제: ID0을 50% CCW 구동하기 위한 프로토콜 0xFF 0xFE 0x00 0x05 0x4E 0x11 0x00 0x13 0x88

15. 피드백 요청 (송신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data	
			Check sum [1byte]	Mode [1byte]
0xFFFF	0x00 ~ 0xFE (0 ~ 254)	0x02	부록 참고	0xA0 (Ping) 0xA1 (위치 피드백) 0xA2 (속도 피드백) 0xA3 (위치제어기 피드백) 0xA4 (속도제어기 피드백) 0xA5 (통신 응답시간 피드백) 0xA6 (외부 감속비 피드백) 0xA7 (제어 On/Off 피드백) 0xA8 (위치제어 모드 피드백) 0xCD (펌웨어 버전 피드백)

예제: ID0을 위치 피드백하기 위한 프로토콜 - 0xFF 0xFE 0x00 0x02 0x5C 0xA1

16. Ping (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data	
			Check sum [1byte]	Mode [1byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x02	부록 참고	0xD0 (Ping)

예제: 0xFF 0xFE 0x00 0x02 0x2D 0xD0 - ID0의 Ping

17. 위치 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	위치방향 [1byte]	위치 [2byte]	속도 [2byte]	전류 [1byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x08	부록 참고	0xD1 (위치제어 피드백)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0xFFFFD (0 ~ 65533) [0.01Degree]	0x0000 ~ 0xFFFFD (0 ~ 65533) [0.1RPM]	0x00 ~ 0xFE (0 ~ 254) [100mA]

예제: 0xFF 0xFE 0x00 0x08 0xA0 0xD1 0x00 0x46 0x40 0x00 0x00 0x00 - ID0은 CCW방향 위치 179.84도, 속도 0.0RPM, 전류 0.0A

18. 속도 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	속도방향 [1byte]	속도 [2byte]	위치 [2byte]	전류 [1byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x08	부록 참고	0xD2 (속도제어 피드백)	0x00 (CCW) 0x01 (CW)	0x0000 ~ 0xFFFFD (0 ~ 65533) [0.1RPM]	0x0000 ~ 0xFFFFD (0 ~ 65533) [0.1Degree]	0x00 ~ 0xFE (0 ~ 254) [100mA]

예제: 0xFF 0xFE 0x00 0x08 0xB7 0xD2 0x00 0x00 0x66 0x78 0x8E 0x02 - ID0은 CCW방향 속도 10.2RPM, 위치 3086.2도, 전류 0.2A

19. 위치제어기 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	위치제어기 Kp [1byte]	위치제어기 Ki [1byte]	위치제어기 Kd [1byte]	위치제어 정격전류 [1byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x06	부록 참고	0xD3 (위치제어기 피드백)	0x01 ~ 0xFE (1 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x01 ~ 정격전류 (1 ~ 정격전류) [100mA] {초기값: 정격전류}

예제: 0xFF 0xFE 0x00 0x06 0x0A 0xD3 0xFE 0xFE 0x00 0x20 - ID0은 위치제어기 Kp=244, Ki=244, Kd=0, 위치제어 정격전류 3.2A

20. 속도제어기 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data					
			Check sum [1byte]	Mode [1byte]	속도제어기 Kp [1byte]	속도제어기 Ki [1byte]	속도제어기 Kd [1byte]	속도제어 정격전류 [1byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x06	부록 참고	0xD4 (속도제어기 피드백)	0x01 ~ 0xFE (1 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x00 ~ 0xFE (0 ~ 254) {초기값: 제품별 상이}	0x01 ~ 정격전류 (1 ~ 정격전류) [100mA] {초기값: 정격전류}

예제: 0xFF 0xFE 0x00 0x06 0x09 0xD4 0xFE 0xFE 0x00 0x20 - ID0은 속도제어기 Kp=244, Ki=244, Kd=0, 속도제어 정격전류 3.2A

21. 통신 응답시간 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	통신 응답시간 [1byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x03	부록 참고	0xD5 (통신 응답시간 피드백)	0x00 ~ 0xFE (0 ~ 254) [100us] {초기값: 0x01}

예제: 0xFF 0xFE 0x00 0x03 0x26 0xD5 0x01 - ID0의 통신 응답시간은 100us

22. 외부 감속비 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	외부 감속비 [2byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x04	부록 참고	0xD6 (감속비 피드백)	0x0001 ~ 0xFFFFD (1 ~ 65533) [0.1ratio] {초기값: 0x000A}

예제: 0xFF 0xFE 0x00 0x04 0x11 0xD6 0x00 0x14 - ID0의 외부 감속비는 2:1

23. 제어 On/Off 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	제어 On/Off [1byte]
0xFFFE	0x00 ~ 0xFE (0 ~ 254)	0x03	부록 참고	0xD7 (제어 On/Off 피드백)	0x00 (On) 0x01 (Off) {초기값: 0x00}

예제: 0xFF 0xFE 0x00 0x03 0x25 0xD7 0x00 - ID0은 제어 On

24. 위치제어 모드 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	절대/상대 위치제어 [1byte]
0xFFFF	0x00 ~ 0xFF (0 ~ 255)	0x03	부록 참고	0xD8 (위치제어 모드 피드백)	0x00 (절대 위치제어) 0x01 (상대 위치제어) (초기값: 0x00)

예제: 0xFF 0xFE 0x00 0x03 0x24 0xD8 0x00 - ID0은 절대 위치제어 모드

25. 펌웨어 버전 피드백 (수신)

Header [2byte]	ID [1byte]	Data 크기 [1byte]	Data		
			Check sum [1byte]	Mode [1byte]	버전 [1byte]
0xFFFF	0x00 ~ 0xFE (0 ~ 254)	0x03	부록 참고	0xFD (펌웨어 버전 피드백)	0x00 ~ 0xFE (0 ~ 254)

예제: 0xFF 0xFE 0x00 0x03 0xFF 0xFD 0x00 - ID0은 펌웨어 버전 0

부 록

참 고	
구 분	설 명
초기 ID	ID: 0x00
Broadcast ID	ID: 0xFF
Check sum	Header, Check sum을 제외한 모든 Packet을 더한 뒤 not연산 ~(Packet 2 + Packet 3 + Packet 5 + ... + Packet N) [1byte]
통신속도 (Baudrate) [bps]	0x00 -> 110
	0x01 -> 300
	0x02 -> 600
	0x03 -> 1,200
	0x04 -> 2,400
	0x05 -> 4,800
	0x06 -> 9,600 (초기값)
	0x07 -> 14,400
	0x08 -> 19,200
	0x09 -> 28,800
	0x0A -> 38,400
	0x0B -> 57,600
	0x0C -> 76,800
	0x0D -> 115,200
제어 불능시	0x0E -> 230,400
	0x0F -> 250,000
	0x10 -> 500,000
	0x11 -> 1,000,000
	14. 제어 방향 설정 변경: 0xFF 0xFE 0x00 0x03 0xEC 0x0F 0x01